

The Bruck–Ryser–Chowla Theorem

Craig Alan Feinstein

Abstract

This entry formalizes the Bruck–Ryser–Chowla theorem for symmetric balanced incomplete block designs. For even order, it proves that the difference between the block size and the index is a perfect square. For odd order, it establishes the existence of a nontrivial integral solution to the associated quadratic equation. The proof proceeds from the incidence-matrix identity, applies a four-square change of variables, uses rational diagonal elimination in the two congruence classes modulo four, and clears denominators to obtain integer solutions.

Contents

1	Preliminaries	2
2	The even-order case	5
3	The fundamental quadratic identity	10
4	The four-square change of variables	16
5	Rational elimination of linear squares	33
6	The case $v = 4w + 1$	69
7	The case $v = 4w - 1$	94
8	Clearing denominators and assembling the theorem	125

```

theory Bruck_Ryser_Chowla imports
  Fishers_Inequality.Fishers_Inequality SumSquares.FourSquares Pell.Pell
begin

```

1 Preliminaries

The Bruck-Ryser-Chowla theorem gives necessary arithmetic conditions for the existence of a symmetric balanced incomplete block design with parameters v , k , and λ . If v is even, then $k - \lambda$ is a perfect square. If v is odd, then a corresponding nontrivial integral quadratic equation has a solution, with its sign determined by the residue of v modulo four. The formalization follows the determinant and rational-elimination proof in D. R. Stinson, *Combinatorial Designs: Constructions and Analysis*, Springer, 2004, Chapter 2 (Symmetric BIBDs), especially Theorems 2.16 and 2.17. Each subsection below identifies the corresponding mathematical step; auxiliary algebra is stated outside the design locale. The proof proceeds through the following stages:

1. **Even order.** The determinant of the incidence-matrix identity is used to prove that the difference between the block size and the index is a perfect square.
2. **The fundamental quadratic identity.** The incidence-matrix equation is converted into an identity involving the linear forms determined by the columns of the incidence matrix.
3. **Four-square change of variables.** A representation of the block-size difference as a sum of four squares is used to transform the variables in blocks of four while preserving the required quadratic identity.
4. **Rational diagonal elimination.** Successive rational substitutions eliminate the linear-square terms. The substitutions preserve the relevant diagonal quadratic form and retain a nontrivial solution.
5. **Order congruent to one modulo four.** The transformed variables produce a nontrivial rational solution of the required plus-sign equation.
6. **Order congruent to three modulo four.** An additional coordinate is introduced, and the same elimination procedure produces a nontrivial rational solution of the required minus-sign equation.

7. **Passage from rational to integer solutions.** Denominators are cleared to turn each nontrivial rational solution into a nontrivial integral solution.
8. **Final odd-order theorem.** The two congruence classes modulo four are combined into the standard Bruck-Ryser-Chowla equation.

Vectors occurring in the quadratic identities are represented as one-column matrices only at the interface with matrix multiplication. The notation below denotes the entry in row i of a one-column matrix while suppressing its constant column index.

```
abbreviation column_entry :: "'a mat  $\Rightarrow$  nat  $\Rightarrow$  'a"
  (infixl "$$_v" 100) where
  "x $$$_v i  $\equiv$  x $$ (i,0)"
```

```
definition column_matrix ::
  "nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat mat" where
  "column_matrix n x =
    mat n 1 ( $\lambda$ (i,j). if j = 0 then x i else 0)"
```

```
lemma dim_row_column_matrix [simp]:
  "dim_row (column_matrix n x) = n"
  unfolding column_matrix_def
  by simp
```

```
lemma dim_col_column_matrix [simp]:
  "dim_col (column_matrix n x) = 1"
  unfolding column_matrix_def
  by simp
```

```
lemma column_matrix_index [simp]:
  assumes "i < n"
  shows "column_matrix n x $$$_v (i) = x i"
  using assms
  unfolding column_matrix_def
  by simp
```

Finite sums of functions are pointwise. This elementary bridge is kept outside the design locale; it is used below to pass between an explicit coordinate sum and the additive finite-sum theorem from the library.

```
lemma finite_sum_functions:
  fixes f :: "'a  $\Rightarrow$  nat  $\Rightarrow$  rat"
  assumes finite: "finite S"
  shows
    " ( $\lambda$ t.  $\sum_{j \in S}. f j t$ )
      =
      ( $\sum_{j \in S}. (\lambda t. f j t)$ )"
  using finite
```

```

proof induction
  case empty
  show ?case
    by (rule ext, simp)
next
case (insert j S)

show ?case
proof (rule ext)
  fix t

  have induction_at_t:
    " $(\lambda t. \sum_{j \in S}. f j t) t$ "
    =
    " $(\sum_{j \in S}. (\lambda t. f j t)) t$ "
    using fun_cong[OF insert.IH, of t]
    .

  show
    " $(\lambda t. \sum_{j \in \text{insert } j S}. f j t) t$ "
    =
    " $(\sum_{j \in \text{insert } j S}. (\lambda t. f j t)) t$ "
    using insert.hyps induction_at_t
    by simp
qed
qed

```

The reconstruction maps are linear in their rational coordinates. The following helper applies the library's additive finite-sum theorem to maps whose additivity is stated pointwise.

```

lemma rat_function_sum:
  fixes F :: "(nat  $\Rightarrow$  rat)  $\Rightarrow$  rat"
  assumes add: " $\bigwedge x y. F (\lambda j. x j + y j) = F x + F y$ "
  shows " $F (\sum_{j \in S}. f j) = (\sum_{j \in S}. F (f j))$ "
proof -
  have additive_F: "additive F"
    unfolding additive_def
  proof (intro allI)
    fix x y :: "nat  $\Rightarrow$  rat"
    have "x + y = ( $\lambda j. x j + y j$ )"
      by (rule ext, simp)
    then show "F (x + y) = F x + F y"
      using add[of x y] by simp
  qed
  show ?thesis
    using Modules.additive.sum[OF additive_F, of f S] .
qed

context ordered_sym_bibd

```

begin

2 The even-order case

For a symmetric design, the replication number equals the block size. Combining this with the standard parameter relation gives the identity needed to simplify the determinant of the incidence-matrix product. This subsection formalizes the determinant argument of Stinson, Theorem 2.16.

```
lemma symmetric_design_parameter_relation:
  "k * (k - 1) =  $\Lambda$  * (v - 1)"
  using necessary_condition_one rep_value_sym
  by simp

lemma block_size_index_identity:
  "k +  $\Lambda$  * (v - 1) = k^2"
proof -
  have
    "k +  $\Lambda$  * (v - 1) =
     k + k * (k - 1)"
    using symmetric_design_parameter_relation
    by simp
  also have
    "... = k^2"
    by (simp add: algebra_simps power2_eq_square)
  finally show ?thesis .
qed
```

The determinant of the incidence matrix multiplied by its transpose is the square of the determinant of the incidence matrix.

```
lemma det_incidence:
  "(det N)^2 = det (N * N^T)"
proof -
  have
    "det (N * N^T) = det N * det N^T"
    by (metis (full_types)
        N_carrier_mat det_mult local.symmetric transpose_carrier_mat)
  also have
    "... = det N * det N"
    using N_carrier_mat det_transpose local.symmetric
    by auto
  also have
    "... = (det N)^2"
    by (simp add: power2_eq_square)
  finally show ?thesis
    by simp
qed
```

The known determinant formula for a regular pairwise balanced design

now simplifies because the design is symmetric and $k + \Lambda * (v - 1) = k^2$.

lemma *symmetric_incidence_determinant_square*:

"(det N)² = k² * (k - Λ)^(v - 1)"

proof -

have

"det (N * N^T) =
(r + Λ * (v - 1)) * (r - Λ)^(v - 1)"

using *determinant_inc_mat_square*

by *simp*

also have

"... =
(k + Λ * (v - 1)) * (k - Λ)^(v - 1)"

using *rep_value_sym*

by *simp*

also have

"... = k² * (k - Λ)^(v - 1)"

using *block_size_index_identity*

by *simp*

finally have

"det (N * N^T) =
k² * (k - Λ)^(v - 1)"

.

then show ?thesis

using *det_incidence*

by *simp*

qed

lemma *determinant_quotient_square*:

"(det N)² / k² = (k - Λ)^(v - 1)"

proof -

have

"(det N)² =
k² * (k - Λ)^(v - 1)"

using *symmetric_incidence_determinant_square*

by *simp*

then have

"(det N)² / k² =
(k² * (k - Λ)^(v - 1)) / k²"

by (*simp add: divide_simps*)

also have

"... =
k² / k² * (k - Λ)^(v - 1)"

by (*simp add: divide_simps*)

also have

"... = 1 * (k - Λ)^(v - 1)"

using *rep_not_zero*

by *fastforce*

also have

"... = (k - Λ)^(v - 1)"

```

      by simp
    finally show ?thesis .
qed

lemma determinant_quotient_power:
  "(det N / k)^2 = (k - Λ)^(v - 1)"
proof -
  have
    "(det N)^2 / k^2 =
     (k - Λ)^(v - 1)"
    using determinant_quotient_square
    by simp
  then show ?thesis
    by (simp add: power_divide)
qed

lemma sqrt_block_difference_power_rational:
  "(sqrt (k - Λ))^(v - 1) ∈ ℚ"
proof -
  have square:
    "(det N / k)^2 = (k - Λ)^(v - 1)"
    using determinant_quotient_power
    by simp

  have
    "sqrt ((k - Λ)^(v - 1)) =
     sqrt ((det N / k)^2)"
    using square
    by simp
  also have
    "... = abs (det N / k)"
    by (metis real_sqrt_abs)
  finally have sqrt_eq:
    "sqrt ((k - Λ)^(v - 1)) =
     abs (det N / k)"
    .

  have power_eq:
    "(sqrt (k - Λ))^(v - 1) =
     sqrt ((k - Λ)^(v - 1))"
    by (simp add: real_sqrt_power)

  have power_abs_eq:
    "(sqrt (k - Λ))^(v - 1) =
     abs (det N / k)"
    using power_eq sqrt_eq
    by simp

  have abs_rational:

```

```

    "abs (det N / k) ∈ ℚ"
  by simp

  show ?thesis
    using power_abs_eq abs_rational
    by simp
qed

```

```

lemma block_size_gt_index:
  "k > Λ"
  using rep_value_sym index_lt_replication
  by auto

```

When the order is even, $v - 1$ is odd. The preceding determinant argument says that an odd power of $\sqrt{\text{real } (k - \Lambda)}$ is rational. Since its square is already rational and nonzero, the square root itself must be rational.

```

lemma bruck_ryser_chowla_even_rat:
  assumes even_v:
    "even v"
  shows
    "sqrt (k - Λ) ∈ ℚ"
proof -
  have difference_rational:
    "k - Λ ∈ ℚ"
    by simp

  have power_rational:
    "(sqrt (k - Λ))^(v - 1) ∈ ℚ"
    using sqrt_block_difference_power_rational
    by blast

  obtain m where v_form:
    "v = 2 * m"
    and m_pos:
    "m > 0"
    using even_v v_non_zero
    by auto

  have exponent:
    "v - 1 = 2 * m - 1"
    using v_form
    by auto

  have odd_power_rational:
    "(sqrt (k - Λ))^(2 * m - 1) ∈ ℚ"
    using power_rational exponent
    by auto

```

```

have quotient_rational:
  "(sqrt (k - Λ))^(2 * m) /
   sqrt (k - Λ) ∈ ℚ"
using odd_power_rational m_pos
  div_by_0 div_less_iff_less_mult mult.commute
  nonzero_mult_div_cancel_left one_div_two_eq_zero
  pos2 power_eq_0_iff power_minus_mult zero_less_diff
by (metis nat_0_less_mult_iff)

have quotient_eq:
  "(sqrt (k - Λ))^(2 * m) /
   sqrt (k - Λ) =
   (k - Λ)^m * (1 / sqrt (k - Λ))"
using block_size_gt_index
by (simp add: power_mult)

have difference_power_rational:
  "(k - Λ)^m ∈ ℚ"
using difference_rational
by (simp add: power_mult Rats_mult_iff)

have inverse_rational:
  "1 / sqrt (k - Λ) ∈ ℚ"
using quotient_rational quotient_eq
  difference_power_rational block_size_gt_index Rats_mult_iff
by force

show ?thesis
  using inverse_rational
  by (simp add: divide_inverse)
qed

The square root of a natural number is either a natural number or ir-
rational. Its rationality therefore implies that it is a natural number, com-
pleting the even-order case.

theorem bruck_ryser_chowla_even:
  assumes even_v:
    "even v"
  shows
    "sqrt (k - Λ) ∈ ℕ"
proof -
  have
    "sqrt (k - Λ) ∈ ℚ"
  using bruck_ryser_chowla_even_rat even_v
  by simp
  then show ?thesis
    using sqrt_nat_or_irrat'
    by blast
qed

```

end

3 The fundamental quadratic identity

The odd-order argument follows the classical “rational equivalence” proof. We first turn the incidence-matrix identity into an equality of quadratic forms. A four-square identity then groups coordinates into blocks of four. Finally, rational elimination removes the remaining linear forms one coordinate at a time. The two possible residues of the order modulo four require slightly different terminal forms.

sublocale *ordered_bibd* $\subseteq$ *ordered_regular_pairwise_balance* $\forall s \ Bs \ \wedge \ r$
by *unfold_locales*

context *ordered_sym_bibd*

begin

We first derive the quadratic identity on which the odd-order argument is based. The all-ones part of the incidence-matrix equation contributes the square of the sum of the coordinates, while the identity-matrix part contributes the sum of their squares. Combining these evaluations with the incidence-matrix identity expresses a sum of squares of incidence linear forms as a diagonal quadratic form.

lemma *all_ones_quadratic_form*:

fixes *x* :: “*rat mat*”

shows

$$\begin{aligned} & "(\sum_{j \in \{0..<v\}}. \sum_{h \in \{0..<v\}}. \\ & \quad (of_nat \ \Lambda \cdot_m J_m \ v) \ \$\$ \ (j,h) * \\ & \quad x \ \$\$_v \ (j) * x \ \$\$_v \ (h)) \\ & = \\ & \quad of_nat \ \Lambda * (\sum_{j \in \{0..<v\}}. x \ \$\$_v \ (j))^2" \end{aligned}$$

proof -

have *J*:

"(of_nat $\Lambda \cdot_m J_m v$) $\$ \$$ (j,h) = of_nat Λ "
if “*j* < *v*” **and** “*h* < *v*”
for *j h*
using *that*
by *simp*

have

$$\begin{aligned} & "(\sum_{j \in \{0..<v\}}. \sum_{h \in \{0..<v\}}. \\ & \quad (of_nat \ \Lambda \cdot_m J_m \ v) \ \$\$ \ (j,h) * \\ & \quad x \ \$\$_v \ (j) * x \ \$\$_v \ (h)) \\ & = \\ & \quad (\sum_{j=0..<v}. \sum_{h=0..<v}. \\ & \quad \quad of_nat \ \Lambda * x \ \$\$_v \ (j) * x \ \$\$_v \ (h))" \end{aligned}$$

```

    using J
    by simp
  also have
    "... =
      (∑ j=0..<v. ∑ h=0..<v.
        of_nat Λ * (x $$$v (j) * x $$$v (h)))"
    by (simp add: algebra_simps)
  also have
    "... =
      (∑ j∈{0..<v}.
        of_nat Λ *
        (∑ h∈{0..<v}. x $$$v (j) * x $$$v (h)))"
    by (simp add: sum_distrib_left)
  also have
    "... =
      of_nat Λ *
      (∑ j∈{0..<v}.
        ∑ h∈{0..<v}. x $$$v (j) * x $$$v (h))"
    by (simp add: sum_distrib_left)
  also have
    "... =
      of_nat Λ *
      ((∑ j∈{0..<v}. x $$$v (j)) *
        (∑ h∈{0..<v}. x $$$v (h)))"
    by (metis sum_product)
  also have
    "... =
      of_nat Λ * (∑ j∈{0..<v}. x $$$v (j))^2"
    by (metis power2_eq_square)
  finally show ?thesis .
qed

```

```

lemma identity_matrix_quadratic_form:
  fixes x :: "rat mat"
  shows
    "(∑ j<v. ∑ h<v.
      (rat_of_nat (r - Λ) ·m 1m v) $$$ (j,h) *
      x $$$v (j) * x $$$v (h))
    =
      rat_of_nat (r - Λ) *
      (∑ j<v. (x $$$v (j))^2)"
  proof -
    have diagonal_entry:
      "rat_of_nat (r - Λ) *
        (if j = h then 1 else 0) *
        x $$$v (j) * x $$$v (h)
      =
        (if j = h
          then rat_of_nat (r - Λ) *

```

```

      x $$$v (j) * x $$$v (h)
    else 0)"
  for j h
  by auto

have
  "( $\sum_{j < v} \sum_{h < v} \text{rat\_of\_nat } (r - \Lambda) \cdot_m 1_m v \text{ } \text{\$ \$ } (j, h) * x \text{ \$ \$ }_v (j) * x \text{ \$ \$ }_v (h))$ 
  =
  ( $\sum_{j < v} \sum_{h < v} \text{rat\_of\_nat } (r - \Lambda) * (\text{if } j = h \text{ then } 1 \text{ else } 0) * x \text{ \$ \$ }_v (j) * x \text{ \$ \$ }_v (h))$ )"
  unfolding one_mat_def
  by simp
also have
  "... =
  ( $\sum_{j < v} \sum_{h < v} \text{if } j = h \text{ then } \text{rat\_of\_nat } (r - \Lambda) * x \text{ \$ \$ }_v (j) * x \text{ \$ \$ }_v (h) \text{ else } 0$ )"
  by (subst diagonal_entry, simp)
also have
  "... =
  ( $\sum_{j < v} \text{rat\_of\_nat } (r - \Lambda) * x \text{ \$ \$ }_v (j) * x \text{ \$ \$ }_v (j)$ )"
  by simp
also have
  "... =
  ( $\sum_{j < v} x \text{ \$ \$ }_v (j) * x \text{ \$ \$ }_v (j)$ )"
  by (simp add: sum_distrib_left mult.assoc)
also have
  "... =
  ( $\text{rat\_of\_nat } (r - \Lambda) * (\sum_{j < v} (x \text{ \$ \$ }_v (j))^2)$ )"
  unfolding power2_eq_square
  by simp
  finally show ?thesis .
qed

```

Adding the preceding two evaluations gives the quadratic form associated with the decomposition of the incidence Gram matrix into its all-ones and identity-matrix components.

```

lemma incidence_diagonal_quadratic_form:
  fixes x :: "rat mat"

```

```

shows "( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}.
  (of\_int (int \Lambda) \cdot_m (J_m v)) \$\$ (j, h) * x \$\$_v (j) * x \$\$_v (h)))$ )
+
  ( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. (of\_int (int (r - \Lambda)) \cdot_m (1_m v)) \$\$
(j, h) *
  x \$\$_v (j) * x \$\$_v (h))) = of\_int (int \Lambda) * (\sum j \in \{0..<v\}. (x \$\$_v
(j)))^2 +
  of\_int (int (r - \Lambda)) * (\sum j \in \{0..<v\}. (x \$\$_v (j))^2)$ )"
proof -
  have lam: "( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. (of\_int (int \Lambda) \cdot_m (J_m v))
  \$\$ (j, h) * x \$\$_v (j) * x \$\$_v (h))) =
  of\_int (int \Lambda) * (\sum j \in \{0..<v\}. (x \$\$_v (j)))^2$ )"
  proof -
    have "(of\_int (int \Lambda) \cdot_m (J_m v :: rat mat)) = (of\_nat \Lambda \cdot_m (J_m v
    :: rat mat))"
    by simp
    then show ?thesis using all_ones_quadratic_form by fastforce
  qed
  have ord: "( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. (of\_int (int (r - \Lambda)) \cdot_m (1_m
v)) \$\$ (j, h) * x \$\$_v (j) * x \$\$_v (h))) =
  of\_int (int (r - \Lambda)) * (\sum j \in \{0..<v\}. (x \$\$_v (j))^2)$ )"
  proof -
    have eq: "(of\_int (int (r - \Lambda)) \cdot_m (1_m v :: rat mat)) = (rat\_of\_nat
(r - \Lambda) \cdot_m (1_m v :: rat mat))"
    by simp
    have "( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. (of\_int (int (r - \Lambda)) \cdot_m (1_m
v)) \$\$ (j, h) * x \$\$_v (j) * x \$\$_v (h))) =
  (\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. (rat\_of\_nat (r - \Lambda) \cdot_m (1_m v))
  \$\$ (j, h) * x \$\$_v (j) * x \$\$_v (h)))$ )"
    using eq by metis
    also have "... = rat\_of\_nat (r - \Lambda) * (\sum j \in \{0..<v\}. (x \$\$_v (j))^2)"
    using identity_matrix_quadratic_form[of x] by (simp add: atLeast0LessThan)
    also have "... = of\_int (int (r - \Lambda)) * (\sum j \in \{0..<v\}. (x \$\$_v (j))^2)"
    by simp
    finally show ?thesis .
  qed
  show ?thesis using lam ord by simp
qed

```

The entries of $N * N^T$ are inner products of rows of the incidence matrix. Expanding those entries converts the matrix quadratic form into a triple sum over rows and columns. The incidence-matrix identity then evaluates that triple sum as the diagonal quadratic form derived above.

```

lemma incidence_gram_quadratic_identity:
  fixes x :: "rat mat"
  shows "( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. (\sum i \in \{0..<v\}.
  of\_int (N \$\$ (j,i)) * of\_int (N \$\$ (h,i)) * x \$\$_v (j) * x \$\$_v (h)))$ )
=
  of\_int (int \Lambda) * (\sum j \in \{0..<v\}. (x \$\$_v (j)))^2 +

```

```

      of_int (int (r -  $\Lambda$ )) * ( $\sum j \in \{0..<v\}. (x \$\$_v (j))^2$ )"
proof -
  have matdef: "( $\sum i \in \{0..<v\} .$ 
    of_int (N  $\$ \$$  (j,i)) * of_int (N  $\$ \$$  (h,i))) = of_int ((N * NT)  $\$ \$$  (j,
h))"
    if "h < v" "j < v" for h j
    using transpose_mat_mult_entries[of "j" "N" "h"]
    local.symmetric that by simp
  have matcond: "of_int ((N * NT)  $\$ \$$  (j, h)) =
    ((of_int (int  $\Lambda$ )  $\cdot_m$  (Jm v) + of_int (int (r -  $\Lambda$ ))  $\cdot_m$  (1m v))  $\$ \$$  (j,
h) :: rat)"
    if "h < v" "j < v" for h j
    using rpbd_incidence_matrix_cond that(1) that(2) by simp
  have sum_eq_rLambda: "( $\sum i \in \{0..<v\}. of_int (N \$ \$ (j,i)) * of_int$ 
(N  $\$ \$$  (h,i))) =
    ((of_int (int  $\Lambda$ )  $\cdot_m$  (Jm v) + of_int (int (r -  $\Lambda$ ))  $\cdot_m$  (1m v))  $\$ \$$  (j,
h) :: rat)"
    if "h < v" "j < v" for h j
  proof -
    have "( $\sum i \in \{0..<v\} . of_int (N \$ \$ (j,i)) * of_int (N \$ \$ (h,i)))$ 
= of_int ((N * NT)  $\$ \$$  (j, h))"
      using matdef[OF that] .
    also have "... = ((of_int (int  $\Lambda$ )  $\cdot_m$  (Jm v) + of_int (int (r -  $\Lambda$ ))
 $\cdot_m$  (1m v))  $\$ \$$  (j, h) :: rat)"
      using matcond[OF that] .
    finally show ?thesis .
  qed
  have "( $\sum i \in \{0..<v\}. of_int (N \$ \$ (j, i)) * of_int (N \$ \$ (h, i))) * x \$ \$_v (j) * x \$ \$_v (h)$ 
=
  ((of_int (int  $\Lambda$ )  $\cdot_m$  (Jm v) + of_int (int (r -  $\Lambda$ ))  $\cdot_m$  (1m v))  $\$ \$$  (j,
h)) *
  x  $\$ \$_v (j) * x \$ \$_v (h)$ " if "h < v" "j < v" for h j
  using sum_eq_rLambda[OF that] by simp
  then have "( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. (\sum i \in \{0..<v\}. of_int (N \$ \$ (j, i)) * of_int (N \$ \$ (h, i))) * x \$ \$_v (j) * x \$ \$_v (h)))$ 
=
  ( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. ((of_int (int  $\Lambda$ )  $\cdot_m$  (Jm v) + of_int (int (r -  $\Lambda$ ))  $\cdot_m$  (1m v))  $\$ \$$  (j,
h)) *
  x  $\$ \$_v (j) * x \$ \$_v (h))$ )" by (intro sum.mono_neutral_cong_right) auto
  also have "... = ( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. ((of_int (int  $\Lambda$ )  $\cdot_m$  (Jm v))  $\$ \$$  (j, h) * x  $\$ \$_v (j) * x \$ \$_v (h)$ ) +
  (of_int (int (r -  $\Lambda$ ))  $\cdot_m$  (1m v))  $\$ \$$  (j, h) * x  $\$ \$_v (j) * x \$ \$_v (h))$ )"
    by (simp add: algebra_simps sum_distrib_left sum_distrib_right)
  also have "... = ( $\sum j \in \{0..<v\}. (\sum h \in \{0..<v\}. ((of_int (int  $\Lambda$ )  $\cdot_m$  (Jm v))  $\$ \$$  (j, h) * x  $\$ \$_v (j) * x \$ \$_v (h))$ ) +
  ( $\sum h \in \{0..<v\}. (of_int (int (r -  $\Lambda$ ))  $\cdot_m$  (1m v))  $\$ \$$  (j, h) *
  x  $\$ \$_v (j) * x \$ \$_v (h))$ )" by (simp add: sum.distrib)$$$$ 
```

```

also have "... = (∑ j ∈ {0..<v}. (∑ h ∈ {0..<v}.
  ((of_int (int Λ) ·m (Jm v)) $$ (j, h) * x $$v (j) * x $$v (h))))
+
  (∑ j ∈ {0..<v}. (∑ h ∈ {0..<v}. (of_int (int (r - Λ)) ·m (1m v))
  $$ (j, h) *
    x $$v (j) * x $$v (h))))"
  by (simp add: sum.distrib)
also have final_equ: "... = of_int (int Λ) * (∑ j ∈ {0..<v}. (x $$v
(j)))^2 +
  of_int (int (r - Λ)) * (∑ j ∈ {0..<v}. (x $$v (j))^2)"
  using incidence_diagonal_quadratic_form by simp
finally show ?thesis .
qed

```

Expanding each squared incidence linear form and reordering the three finite sums gives the quadratic form evaluated in $(\sum j = 0..<v. \sum h = 0..<v. (\sum i = 0..<v. \text{rat_of_int } (N \text{ } \$\$ (j, i)) * \text{rat_of_int } (N \text{ } \$\$ (h, i))) * ?x \text{ } \$\$_v j * ?x \text{ } \$\$_v h) = \text{rat_of_int } (\text{int } \Lambda) * (\text{sum } ((\$$_v) ?x) \{0..<v\})^2 + \text{rat_of_int } (\text{int } (r - \Lambda)) * (\sum j = 0..<v. (?x \text{ } \$$_v j)^2)$. Symmetry supplies $r = k$. The resulting identity says that the sum of the squares of the incidence linear forms equals the index times the square of the coordinate sum, plus the block-size difference times the sum of the coordinate squares. This is the starting point for the four-square transformations used in both odd congruence cases.

```

lemma brc_x_equation:
  fixes x :: "rat mat"
  shows "(∑ i ∈ {0..<v}. (∑ h ∈ {0..<v}. of_int (N $$ (h,i)) * x $$v
(h))^2) =
  of_int (int Λ) * (∑ j ∈ {0..<v}. (x $$v (j)))^2 +
  of_int (int (k - Λ)) * (∑ j ∈ {0..<v}. (x $$v (j))^2)"
proof -
  have r_eq_k: "r = k" using rep_value_sym by simp
  have "(∑ i ∈ {0..<v}. (∑ h ∈ {0..<v}. of_int (N $$ (h,i)) * x $$v (h))^2)
  =
    (∑ i ∈ {0..<v}. (∑ j ∈ {0..<v}. of_int (N $$ (j, i)) *
    x $$v (j)) * (∑ h ∈ {0..<v}. of_int (N $$ (h,i)) * x $$v (h))))"
    by (simp add: power2_eq_square)
  also have "... = (∑ i ∈ {0..<v}. (∑ j ∈ {0..<v}. (∑ h ∈ {0..<v}.
    (of_int (N $$ (j, i)) * x $$v (j)) * (of_int (N $$ (h,i)) * x $$v
(h))))))"
    by (metis (no_types) sum_product)
  also have "... = (∑ i ∈ {0..<v}. (∑ j ∈ {0..<v}. (∑ h ∈ {0..<v}.
    of_int (N $$ (j, i)) * (x $$v (j) * of_int (N $$ (h,i))) * x $$v (h))))"
    by (simp add: algebra_simps)
  also have "... = (∑ i ∈ {0..<v}. (∑ j ∈ {0..<v}. (∑ h ∈ {0..<v}.
    of_int (N $$ (j, i)) * (of_int (N $$ (h,i)) * x $$v (j)) * x $$v (h))))"
    by (metis (no_types, lifting) mult_of_int_commute sum.cong)
  also have "... = (∑ i ∈ {0..<v}. (∑ j ∈ {0..<v}. (∑ h ∈ {0..<v}.

```

```

    of_int (N $$ (j, i)) * of_int (N $$ (h, i)) * x $$v (j) * x $$v (h))))"
  by (simp add: algebra_simps)
also have "... = (∑ j ∈ {0..v}. (∑ h ∈ {0..v}. (∑ i ∈ {0..v}.
  of_int (N $$ (j, i)) * of_int (N $$ (h, i)) * x $$v (j) * x $$v (h))))"
  using sum_reorder_triple[of "λ i j h . of_int (N $$ (j, i)) * of_int
(N $$ (h, i)) *
  x $$v (j) * x $$v (h)" "{0..v" "{0..v" "{0..v"}]
  by blast
also have "... = (∑ j ∈ {0..v}. (∑ h ∈ {0..v}. (∑ i ∈ {0..v}.
  of_int (N $$ (j, i)) * of_int (N $$ (h, i)) * x $$v (j) * x $$v (h))))"
  by (simp add: algebra_simps sum_distrib_left sum_distrib_right)
also have "... = of_int (int Λ) * (∑ j ∈ {0..v}. (x $$v (j)))2 +
  of_int (int (r - Λ)) * (∑ j ∈ {0..v}. (x $$v (j)))2"
  using incidence_gram_quadratic_identity by simp
also have "... = of_int (int Λ) * (∑ j ∈ {0..v}. (x $$v (j)))2 +
  of_int (int (k - Λ)) * (∑ j ∈ {0..v}. (x $$v (j)))2"
  using r_eq_k by simp
finally show ?thesis .
qed

end

```

4 The four-square change of variables

The following maps are the rational change of coordinates associated with Euler's four-square identity. The forward map multiplies a four-vector by the quaternionic matrix determined by (a, b, c, d) ; the inverse map uses its transpose and divides by $a^2 + b^2 + c^2 + d^2$. Consequently the squared norm is multiplied by that sum of four squares. The coefficient tuple is an argument of the maps rather than part of the transformed vector.

```

fun y_of :: "(nat × nat × nat × nat) ×
  (rat × rat × rat × rat)) ⇒
  (rat × rat × rat × rat)" where
  "y_of((a, b, c, d), (x0, x1, x2, x3)) =
    (of_nat a * x0 + of_nat b * x1 + of_nat c * x2 + of_nat d * x3,
     - of_nat b * x0 + of_nat a * x1 - of_nat d * x2 + of_nat c * x3,
     - of_nat c * x0 + of_nat d * x1 + of_nat a * x2 - of_nat b * x3,
     - of_nat d * x0 - of_nat c * x1 + of_nat b * x2 + of_nat a * x3)"

fun y_inv_of ::
  "(nat × nat × nat × nat) ×
  (rat × rat × rat × rat)) ⇒
  (rat × rat × rat × rat)"
where
  "y_inv_of ((a, b, c, d), (y0, y1, y2, y3)) =
    (((of_nat a)*y0 - (of_nat b)*y1 - (of_nat c)*y2 - (of_nat d)*y3) /
     of_nat (a2 + b2 + c2 + d2),
     ((of_nat b)*y0 + (of_nat a)*y1 + (of_nat d)*y2 - (of_nat c)*y3) /

```

$$\frac{\text{of_nat } (a^2 + b^2 + c^2 + d^2), ((\text{of_nat } c)*y0 + (\text{of_nat } a)*y2 + (\text{of_nat } b)*y3 - (\text{of_nat } d)*y1) / \text{of_nat } (a^2 + b^2 + c^2 + d^2), ((\text{of_nat } d)*y0 + (\text{of_nat } c)*y1 + (\text{of_nat } a)*y3 - (\text{of_nat } b)*y2) / \text{of_nat } (a^2 + b^2 + c^2 + d^2))}{}$$

Provided that the sum of the four coefficient squares is nonzero, the inverse transformation is a right inverse of the forward four-square transformation. This allows coordinates introduced by the four-square change of variables to be converted back into the original variables.

```

lemma four_square_transform_inverse:
  fixes a b c d :: nat
  fixes y0 y1 y2 y3 :: rat
  assumes nz: "a^2 + b^2 + c^2 + d^2 ≠ 0"
  shows
    "y_of ((a, b, c, d),
      y_inv_of ((a, b, c, d), (y0, y1, y2, y3))) =
      (y0, y1, y2, y3)"
proof -
  have cast_nz:
    "(of_nat (a^2 + b^2 + c^2 + d^2) :: rat) ≠ 0"
  using nz
  by (simp only: of_nat_0_eq_iff)

  have denominator_eq:
    "rat_of_nat a * rat_of_nat a +
      (rat_of_nat b * rat_of_nat b +
        (rat_of_nat c * rat_of_nat c +
          rat_of_nat d * rat_of_nat d)) =
      (of_nat (a^2 + b^2 + c^2 + d^2) :: rat)"
  by (simp only:
    of_nat_add
    of_nat_mult
    power2_eq_square
    add.assoc)

  have denominator_nz:
    "rat_of_nat a * rat_of_nat a +
      (rat_of_nat b * rat_of_nat b +
        (rat_of_nat c * rat_of_nat c +
          rat_of_nat d * rat_of_nat d)) ≠ 0"
  using cast_nz denominator_eq
  by simp

  show ?thesis
  using denominator_nz
  by (simp add:
    add_divide_distrib [symmetric]
    diff_divide_distrib [symmetric])

```

```

      power2_eq_square
      algebra_simps;
    simp add: distrib_left [symmetric])
qed

```

The forward four-square transformation multiplies the squared norm of a four-coordinate vector by the sum of the four coefficient squares. This is the algebraic identity that permits the coordinates of the quadratic form to be processed in blocks of four.

```

lemma four_square_norm_identity:
  fixes a b c d :: nat
  fixes x0 x1 x2 x3 :: rat
  assumes n_def: "n = a^2 + b^2 + c^2 + d^2"
  shows
    "(fst (y_of ((a,b,c,d),(x0,x1,x2,x3))))^2 +
     (fst (snd (y_of ((a,b,c,d),(x0,x1,x2,x3))))^2 +
     (fst (snd (snd (y_of ((a,b,c,d),(x0,x1,x2,x3))))))^2 +
     (snd (snd (snd (y_of ((a,b,c,d),(x0,x1,x2,x3))))))^2
    =
     of_nat n * (x0^2 + x1^2 + x2^2 + x3^2)"
  unfolding n_def
  by (simp add: algebra_simps power2_eq_square)

```

```

context ordered_sym_bibd
begin

```

```

definition t_of :: "rat mat  $\Rightarrow$  rat" where
  "t_of x = ( $\sum_{j \in \{0..<v\}} x \cdot x_v(j)$ )"

```

```

lemma brc_v_eq_4w_plus_1:
  assumes "v mod 4 = 1"
  shows " $\exists w. v = 4 * w + 1$ "
proof
  show "v = 4 * (v div 4) + 1"
    using assms
    by (metis div_mult_mod_eq mult.commute)
qed

```

Specializing the four-square norm identity to a representation $k - \Lambda = a^2 + b^2 + c^2 + d^2$ shows that the transformed squared norm is exactly $k - \Lambda$ times the original squared norm.

```

lemma four_square_norm_identity_block_difference:
  fixes a b c d :: nat
  fixes x0 x1 x2 x3 :: rat
  assumes abcd: "k -  $\Lambda$  = a^2 + b^2 + c^2 + d^2"
  shows
    "(fst (y_of ((a,b,c,d),(x0,x1,x2,x3))))^2 +
     (fst (snd (y_of ((a,b,c,d),(x0,x1,x2,x3))))^2 +
     (fst (snd (snd (y_of ((a,b,c,d),(x0,x1,x2,x3))))))^2 +
     (snd (snd (snd (y_of ((a,b,c,d),(x0,x1,x2,x3))))))^2

```

```

      (snd (snd (snd (y_of ((a,b,c,d),(x0,x1,x2,x3)))))^2
    =
      of_nat (k - Λ) * (x0^2 + x1^2 + x2^2 + x3^2)"
using four_square_norm_identity[of "k - Λ" a b c d x0 x1 x2 x3] abcd
by simp

```

Writing the four transformed coordinates separately, their squared sum equals $k - \Lambda$ times the squared sum of the corresponding four original coordinates.

```

lemma four_square_block_sum_identity:
  fixes a b c d :: nat
  fixes x0 x1 x2 x3 :: rat
  assumes abcd: "k - Λ = a^2 + b^2 + c^2 + d^2"
  shows
    "of_nat (k - Λ) * (x0^2 + x1^2 + x2^2 + x3^2)
    =
      (fst (y_of ((a,b,c,d),(x0,x1,x2,x3))))^2 +
      (fst (snd (y_of ((a,b,c,d),(x0,x1,x2,x3)))))^2 +
      (fst (snd (snd (y_of ((a,b,c,d),(x0,x1,x2,x3)))))^2 +
      (snd (snd (snd (y_of ((a,b,c,d),(x0,x1,x2,x3)))))^2"
using four_square_norm_identity_block_difference[OF abcd, of x0 x1 x2
x3]
by simp

```

Applying the four-coordinate identity to the entries numbered $4 * h$, $4 * h + 1$, $4 * h + 2$, and $4 * h + 3$ gives the norm-preservation identity for the h -th block of a matrix column.

```

lemma four_square_indexed_block_identity:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  assumes abcd: "k - Λ = a^2 + b^2 + c^2 + d^2"
  shows
    "of_nat (k - Λ) *
      ((x $$v (4*h))^2 +
      (x $$v (4*h + 1))^2 +
      (x $$v (4*h + 2))^2 +
      (x $$v (4*h + 3))^2)
    =
      (fst (y_of ((a,b,c,d),
      (x $$v (4*h), x $$v (4*h + 1), x $$v (4*h + 2), x $$v (4*h + 3))))))^2
    +
      (fst (snd (y_of ((a,b,c,d),
      (x $$v (4*h), x $$v (4*h + 1), x $$v (4*h + 2), x $$v (4*h + 3))))))^2
    +
      (fst (snd (snd (y_of ((a,b,c,d),
      (x $$v (4*h), x $$v (4*h + 1), x $$v (4*h + 2), x $$v (4*h + 3))))))^2
    +
      (snd (snd (snd (y_of ((a,b,c,d),
      (x $$v (4*h), x $$v (4*h + 1), x $$v (4*h + 2), x $$v (4*h + 3))))))^2"

```

```

using four_square_block_sum_identity[OF abcd,
  of "x $$$v (4*h)"
    "x $$$v (4*h + 1)"
    "x $$$v (4*h + 2)"
    "x $$$v (4*h + 3)"]
by simp

definition x_block_sqsum :: "rat mat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "x_block_sqsum x h =
    (x $$$v (4*h))^2 +
    (x $$$v (4*h + 1))^2 +
    (x $$$v (4*h + 2))^2 +
    (x $$$v (4*h + 3))^2"

definition y_block_sqsum ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
  rat mat  $\Rightarrow$  nat  $\Rightarrow$  rat"
where
  "y_block_sqsum a b c d x h =
    (case y_of ((a,b,c,d),
      (x $$$v (4*h),
       x $$$v (4*h + 1),
       x $$$v (4*h + 2),
       x $$$v (4*h + 3))))
    of (y0,y1,y2,y3)  $\Rightarrow$ 
      y0^2 + y1^2 + y2^2 + y3^2)"

lemma y_block_sqsum_identity:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  assumes abcd:
    "k -  $\Lambda$  = a^2 + b^2 + c^2 + d^2"
  shows
    "of_nat (k -  $\Lambda$ ) * x_block_sqsum x h =
      y_block_sqsum a b c d x h"
  using four_square_indexed_block_identity[OF abcd, of x h]
  unfolding x_block_sqsum_def y_block_sqsum_def
  by simp

lemma y_blocks_sqsum_identity:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  assumes abcd: "k -  $\Lambda$  = a^2 + b^2 + c^2 + d^2"
  shows
    "of_nat (k -  $\Lambda$ ) * ( $\sum_{h \in \{0..<w\}} x\_block\_sqsum\ x\ h$ )
      =
      ( $\sum_{h \in \{0..<w\}} y\_block\_sqsum\ a\ b\ c\ d\ x\ h$ )"
  proof -
    have " $\bigwedge h. h \in \{0..<w\} \implies$ "

```

```

    of_nat (k -  $\Lambda$ ) * x_block_sqsum x h =
    y_block_sqsum a b c d x h"
    using y_block_sqsum_identity[OF abcd] by simp
  then have
    "(\sum h \in \{0..<w\}. of_nat (k -  $\Lambda$ ) * x_block_sqsum x h)
    =
    (\sum h \in \{0..<w\}. y_block_sqsum a b c d x h)"
    by simp
  then show ?thesis
    by (simp add: sum_distrib_left)
qed

```

```

lemma brc_x_sqsum_split_last:
  fixes x :: "rat mat"
  assumes v_form: "v = 4 * w + 1"
  shows
    "(\sum j \in \{0..<v\}. (x $$$v (j)))^2)
    =
    (\sum j \in \{0..<4*w\}. (x $$$v (j)))^2) + (x $$$v (4*w))^2"
  proof -
    have "v = Suc (4*w)"
      using v_form by simp
    then show ?thesis
      by (simp add: sum.atLeast0_lessThan_Suc)
  qed

```

```

lemma brc_x_sum_split_last:
  fixes x :: "rat mat"
  assumes v_form: "v = 4 * w + 1"
  shows
    "(\sum j \in \{0..<v\}. x $$$v (j))
    =
    (\sum j \in \{0..<4*w\}. x $$$v (j)) + x $$$v (4*w)"
  proof -
    have "v = Suc (4*w)"
      using v_form by simp
    then show ?thesis
      by (simp add: sum.atLeast0_lessThan_Suc)
  qed

```

```

lemma brc_x_equation_split_last_coordinate:
  fixes x :: "rat mat"
  assumes v_form: "v = 4 * w + 1"
  shows
    "(\sum i \in \{0..<v\}.
      (\sum h \in \{0..<v\}. of_int (N $$$ (h,i)) * x $$$v (h))^2)
    =
    of_nat  $\Lambda$  *
    ((\sum j \in \{0..<4*w\}. x $$$v (j)))

```

```

      + x $$$v (4*w))^2
+
of_nat (k - Λ) *
  ((∑ j∈{0..<4*w}. (x $$$v (j))^2)
   + (x $$$v (4*w))^2)"
proof -
  have eq:
    "(∑ i∈{0..v (h))^2)
    =
    of_nat Λ * (∑ j∈{0..v (j))^2
    +
    of_nat (k - Λ) * (∑ j∈{0..v (j))^2)"
    using brc_x_equation[of x]
    by simp

  have sq:
    "(∑ j∈{0..v (j))^2)
    =
    (∑ j∈{0..<4*w}. (x $$$v (j))^2)
    + (x $$$v (4*w))^2"
    using brc_x_sqsum_split_last[OF v_form] .

  have lin:
    "(∑ j∈{0..v (j))
    =
    (∑ j∈{0..<4*w}. x $$$v (j))
    + x $$$v (4*w)"
    using brc_x_sum_split_last[OF v_form] .

  show ?thesis
    using eq sq lin
    by simp
qed

definition x_head_sum :: "rat mat ⇒ nat ⇒ rat" where
  "x_head_sum x w = (∑ j∈{0..<4*w}. x $$$v (j))"

definition x_head_sqsum :: "rat mat ⇒ nat ⇒ rat" where
  "x_head_sqsum x w = (∑ j∈{0..<4*w}. (x $$$v (j))^2)"

definition x_last :: "rat mat ⇒ nat ⇒ rat" where
  "x_last x w = x $$$v (4*w)"

lemma brc_x_equation_head_last:
  fixes x :: "rat mat"
  assumes v_form: "v = 4 * w + 1"
  shows
    "(∑ i∈{0..

```

```

      (∑ h∈{0..<v}. of_int (N $$$ (h,i)) * x $$$v (h))^2)
    =
    of_nat Λ * (x_head_sum x w + x_last x w)^2
  +
    of_nat (k - Λ) *
      (x_head_sqsum x w + (x_last x w)^2)"
  using brc_x_equation_split_last_coordinate[OF v_form, of x]
  unfolding x_head_sum_def x_head_sqsum_def x_last_def
  by simp

definition y_blocks_sqsum :: "nat ⇒ nat ⇒ nat ⇒ nat ⇒ rat mat ⇒ nat
⇒ rat" where
  "y_blocks_sqsum a b c d x w =
    (∑ h∈{0..<w}. y_block_sqsum a b c d x h)"

lemma y_blocks_sqsum_scaled:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  assumes abcd: "k - Λ = a^2 + b^2 + c^2 + d^2"
  shows
    "of_nat (k - Λ) * (∑ h∈{0..<w}. x_block_sqsum x h)
    =
    y_blocks_sqsum a b c d x w"
  using y_blocks_sqsum_identity[OF abcd, of x w]
  unfolding y_blocks_sqsum_def
  by simp

lemma brc_x_equation_transformed_from_block_partition:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  assumes v_form: "v = 4 * w + 1"
  assumes abcd: "k - Λ = a^2 + b^2 + c^2 + d^2"
  assumes head_blocks:
    "x_head_sqsum x w = (∑ h∈{0..<w}. x_block_sqsum x h)"
  shows
    "(∑ i∈{0..<v}.
      (∑ h∈{0..<v}. of_int (N $$$ (h,i)) * x $$$v (h))^2)
    =
    of_nat Λ * (x_head_sum x w + x_last x w)^2
  +
    y_blocks_sqsum a b c d x w
  +
    of_nat (k - Λ) * (x_last x w)^2"
  proof -
    have split_eq:
      "(∑ i∈{0..<v}.
        (∑ h∈{0..<v}. of_int (N $$$ (h,i)) * x $$$v (h))^2)
      =
      of_nat Λ * (x_head_sum x w + x_last x w)^2"

```

```

+
of_nat (k -  $\Lambda$ ) *
  (x_head_sqsum x w + (x_last x w)^2)"
using brc_x_equation_head_last[OF v_form, of x] .

have block_eq:
  "of_nat (k -  $\Lambda$ ) * x_head_sqsum x w =
   y_blocks_sqsum a b c d x w"
proof -
  have "of_nat (k -  $\Lambda$ ) * x_head_sqsum x w =
        of_nat (k -  $\Lambda$ ) * ( $\sum_{h \in \{0..<w\}} x\_block\_sqsum\ x\ h$ )"
    using head_blocks by simp
  also have "... = y_blocks_sqsum a b c d x w"
    using y_blocks_sqsum_scaled[OF abcd, of x w] .
  finally show ?thesis .
qed

show ?thesis
  using split_eq block_eq
  by (simp add: algebra_simps)
qed

lemma sum_four_block_Suc:
  fixes f :: "nat  $\Rightarrow$  rat"
  shows " $(\sum_{j = 0..<4 + n * 4} f\ j)$ 
    =
    f (n * 4) + f (Suc (n * 4)) +
    f (Suc (Suc (n * 4))) + f (3 + n * 4) +
     $(\sum_{j = 0..<n * 4} f\ j)$ "
proof -
  have " $(\sum_{j = 0..<4 + n * 4} f\ j)$ 
    =  $(\sum_{j = 0..<n * 4} f\ j)$  +  $(\sum_{j = n * 4..<4 + n * 4} f\ j)$ "
    by (simp add: sum.atLeastLessThan_concat)
  also have " $(\sum_{j = n * 4..<4 + n * 4} f\ j)$ 
    = f (n * 4) + f (Suc (n * 4)) + f (Suc (Suc (n * 4))) + f (3 + n
* 4)"
    by (simp add: numeral_eq_Suc)
  finally show ?thesis
    by (simp add: algebra_simps)
qed

lemma x_head_sqsum_as_blocks:
  fixes x :: "rat mat"
  shows "x_head_sqsum x w =  $(\sum_{h \in \{0..<w\}} x\_block\_sqsum\ x\ h)$ "
proof (induct w)
  case 0
  show ?case
    unfolding x_head_sqsum_def x_block_sqsum_def
    by simp

```

```

next
  case (Suc n)
  have block_partition:
    "x_head_sqsum x n = ( $\sum_{h \in \{0..<n\}} x\_block\_sqsum\ x\ h$ )"
    using Suc.hyps by simp

  have split:
    "x_head_sqsum x (Suc n)
    =
    x_head_sqsum x n + x_block_sqsum x n"
    unfolding x_head_sqsum_def x_block_sqsum_def
    using sum_four_block_Suc[of " $\lambda j. (x\ \$\$_v\ (j))^2$ " n]
    by (simp add: algebra_simps)

  show ?case
    using block_partition split
    by simp
qed

lemma brc_x_equation_transformed:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  assumes v_form: "v = 4 * w + 1"
  assumes abcd: " $k - \Lambda = a^2 + b^2 + c^2 + d^2$ "
  shows
    " $(\sum_{i \in \{0..<v\}} (\sum_{h \in \{0..<v\}} of\_int\ (N\ \$\$ (h,i)) * x\ \$\$_v\ (h))^2)$ 
    =
    of_nat  $\Lambda$  * (x_head_sum x w + x_last x w)^2
    +
    y_blocks_sqsum a b c d x w
    +
    of_nat (k -  $\Lambda$ ) * (x_last x w)^2"
  using brc_x_equation_transformed_from_block_partition[
    OF v_form abcd x_head_sqsum_as_blocks]
  by simp

definition brc_y0 :: "rat mat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "brc_y0 x w = x_head_sum x w + x_last x w"

definition brc_yv :: "rat mat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "brc_yv x w = x_last x w"

definition brc_L :: "rat mat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "brc_L x i =
    ( $\sum_{h \in \{0..<v\}} of\_int\ (N\ \$\$ (h,i)) * x\ \$\$_v\ (h)$ )"

lemma brc_x_equation_in_linear_forms:
  fixes a b c d :: nat

```

```

fixes x :: "rat mat"
assumes v_form: "v = 4 * w + 1"
assumes abcd: "k -  $\Lambda$  = a2 + b2 + c2 + d2"
shows
  "(\sum i \in \{0..<v\}. (brc_L x i)^2)
   =
   of_nat  $\Lambda$  * (brc_y0 x w)^2
   +
   y_blocks_sqsum a b c d x w
   +
   of_nat (k -  $\Lambda$ ) * (brc_yv x w)^2"
using brc_x_equation_transformed[OF v_form abcd, of x]
unfolding brc_L_def brc_y0_def brc_yv_def
by simp

```

```

lemma brc_x_equation_with_auxiliary_coordinate:
  fixes x :: "rat mat"
  fixes xv1 :: rat
  shows
    "(\sum i \in \{0..<v\}.
      (\sum h \in \{0..<v\}.
        of_int (N $$ (h,i)) * x $$v (h))^2
      + of_int (int (k -  $\Lambda$ )) * xv1^2)
     =
     of_int (int  $\Lambda$ ) *
     (\sum j \in \{0..<v\}. x $$v (j))^2
     + of_int (int (k -  $\Lambda$ )) *
     ((\sum j \in \{0..<v\}. (x $$v (j))^2) + xv1^2)"
  proof -
    have base:
      "(\sum i \in \{0..<v\}.
        (\sum h \in \{0..<v\}.
          of_int (N $$ (h,i)) * x $$v (h))^2)
       =
       of_int (int  $\Lambda$ ) *
       (\sum j \in \{0..<v\}. x $$v (j))^2
       + of_int (int (k -  $\Lambda$ )) *
       (\sum j \in \{0..<v\}. (x $$v (j))^2)"
      using brc_x_equation[of x] .
    show ?thesis
      using base
      by (simp add: algebra_simps)
  qed

```

```

definition brc_extend_x :: "rat mat  $\Rightarrow$  rat  $\Rightarrow$  rat mat" where
  "brc_extend_x x xv1 =
   column_matrix (v + 1)
   ( $\lambda$ i. if i < v then x $$v (i) else xv1)"

```

```

lemma brc_extend_x_initial:
  assumes "i < v"
  shows "brc_extend_x x xv1 $$$v (i) = x $$$v (i)"
  using assms
  unfolding brc_extend_x_def
  by simp

```

```

lemma brc_extend_x_last:
  shows "brc_extend_x x xv1 $$$v (v) = xv1"
  unfolding brc_extend_x_def
  by simp

```

```

lemma brc_extend_x_sqsum:
  "(\sum i \in \{0..<v + 1\}.
    (brc_extend_x x xv1 $$$v (i))^2)
  =
  (\sum i \in \{0..<v\}. (x $$$v (i))^2) + xv1^2"

```

```

proof -
  have split:
    "{0..<v + 1} = {0..<v} \cup {v}"
  by auto

```

```

  have disj:
    "{0..<v} \cap {v} = {}"
  by auto

```

```

  have
    "(\sum i \in \{0..<v + 1\}.
      (brc_extend_x x xv1 $$$v (i))^2)
    =
    (\sum i \in \{0..<v\}.
      (brc_extend_x x xv1 $$$v (i))^2)
      + (brc_extend_x x xv1 $$$v (v))^2"
    using split disj
    by (simp add: sum.union_disjoint)

```

```

  also have
    "... =
    (\sum i \in \{0..<v\}. (x $$$v (i))^2) + xv1^2"
    using brc_extend_x_initial brc_extend_x_last
    by (intro arg_cong2[where f="(+)"]) auto

```

```

  finally show ?thesis .
qed

```

```

lemma brc_v_plus_one_four_blocks:
  assumes v_form: "v = 4 * w - 1"
  assumes w_pos: "0 < w"

```

```

shows "v + 1 = 4 * w"
using v_form w_pos
by simp

lemma sum_four_blocks:
  fixes f :: "nat ⇒ rat"
  shows
    "(\sum i ∈ {0..<4*w}. f i)
     =
     (\sum h ∈ {0..<w}.
      (f (4*h) + f (4*h+1) + f (4*h+2) + f (4*h+3)))"
proof (induction w)
  case 0
  then show ?case
    by simp
next
  case (Suc w)

  have split:
    "{0..<4 * Suc w} =
     {0..<4*w} ∪ {4*w, 4*w+1, 4*w+2, 4*w+3}"
    by auto

  have disj:
    "{0..<4*w} ∩ {4*w, 4*w+1, 4*w+2, 4*w+3} = {}"
    by auto

  show ?case
    unfolding split
    using disj Suc.IH
    by (simp add: sum.union_disjoint algebra_simps)
qed

lemma brc_x_equation_for_extended_vector:
  fixes x :: "rat mat"
  fixes xv1 :: rat
  shows
    "(\sum i ∈ {0..<v}.
     (\sum h ∈ {0..<v}.
      of_int (N $$ (h,i)) * x $$v (h))^2
     + of_int (int (k - Λ)) * xv1^2
    =
     of_int (int Λ) *
     (\sum j ∈ {0..<v}. x $$v (j))^2
     + of_int (int (k - Λ)) *
     (\sum j ∈ {0..<v + 1}.
      (brc_extend_x x xv1 $$v (j))^2)"
proof -
  have ext:

```

```

"( $\sum j \in \{0..<v + 1\}.$ 
  ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (j))$ )2
=
( $\sum j \in \{0..<v\}.$  ( $x \$\$_v (j)$ )2) +  $\text{xv1}^2$ "
using  $\text{brc\_extend\_x\_sqsum}[\text{of } x \text{ xv1}]$  .

have base:
"( $\sum i \in \{0..<v\}.$ 
  ( $\sum h \in \{0..<v\}.$ 
    of_int ( $N \$\$ (h,i)$ ) *  $x \$\$_v (h)$ )2
  + of_int ( $\text{int } (k - \Lambda)$ ) *  $\text{xv1}^2$ 
  =
    of_int ( $\text{int } \Lambda$ ) *
      ( $\sum j \in \{0..<v\}.$   $x \$\$_v (j)$ )2
  + of_int ( $\text{int } (k - \Lambda)$ ) *
      ( $\sum j \in \{0..<v\}.$  ( $x \$\$_v (j)$ )2) +  $\text{xv1}^2$ )"
using  $\text{brc\_x\_equation\_with\_auxiliary\_coordinate}[\text{of } x \text{ xv1}]$  .

show ?thesis
  using base ext
  by simp
qed

lemma  $\text{brc\_extend\_x\_sqsum\_blocks}$ :
  assumes  $v\_form$ : " $v = 4 * w - 1$ "
  assumes  $w\_pos$ : " $0 < w$ "
  shows
    " $(\sum i \in \{0..<v + 1\}.$ 
      ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (i)$ )2)
    =
      ( $\sum h \in \{0..<w\}.$ 
        ( $(\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h))$ )2 +
          ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h+1)$ )2 +
          ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h+2)$ )2 +
          ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h+3)$ )2))"
  proof -
    have size:
      " $v + 1 = 4 * w$ "
      using  $\text{brc\_v\_plus\_one\_four\_blocks}[0F \text{ } v\_form \text{ } w\_pos]$  .

    have blocks:
      " $(\sum i \in \{0..<4*w\}.$ 
        ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (i)$ )2)
      =
        ( $\sum h \in \{0..<w\}.$ 
          ( $(\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h))$ )2 +
            ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h+1)$ )2 +
            ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h+2)$ )2 +
            ( $\text{brc\_extend\_x } x \text{ xv1 } \$\$_v (4*h+3)$ )2))"

```

```

using sum_four_blocks[
  of "λi. (brc_extend_x x xv1 $$v (i))^2" w] .

from blocks show ?thesis
  using size
  by simp
qed

lemma brc_extend_x_transformed_blocks:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  fixes xv1 :: rat
  assumes v_form: "v = 4 * w - 1"
  assumes w_pos: "0 < w"
  assumes four_sq:
    "k - Λ = a^2 + b^2 + c^2 + d^2"
  shows
    "of_nat (k - Λ) *
      (∑ i∈{0..v (i))^2)
    =
      (∑ h∈{0..v (i))^2)
    =
      (∑ h∈{0..v (i))^2)
      =
        (∑ h∈{0..v (4*h))^2 +
           (brc_extend_x x xv1 $$v (4*h+1))^2 +
           (brc_extend_x x xv1 $$v (4*h+2))^2 +
           (brc_extend_x x xv1 $$v (4*h+3))^2))"
    using brc_extend_x_sqsum_blocks[
      OF v_form w_pos, of x xv1] .

  show ?thesis
    using split
    unfolding x_block_sqsum_def
    by simp
qed

```

```

have block_transform:
  "\h. h ∈ {0..<w} ⇒
    of_nat (k - Λ) *
      x_block_sqsum (brc_extend_x x xv1) h
    =
      y_block_sqsum a b c d
        (brc_extend_x x xv1) h"
using y_block_sqsum_identity[
  OF four_sq, of "brc_extend_x x xv1"]
by simp

have
  "of_nat (k - Λ) *
    (∑ i∈{0..<v + 1}.
      (brc_extend_x x xv1 $$$ (i))^2)
  =
    of_nat (k - Λ) *
    (∑ h∈{0..<w}.
      x_block_sqsum (brc_extend_x x xv1) h)"
using blocks
by simp

also have
  "... =
    (∑ h∈{0..<w}.
      of_nat (k - Λ) *
        x_block_sqsum (brc_extend_x x xv1) h)"
by (simp add: sum_distrib_left)

also have
  "... =
    (∑ h∈{0..<w}.
      y_block_sqsum a b c d
        (brc_extend_x x xv1) h)"
using block_transform
by simp

finally show ?thesis .
qed

lemma brc_x_equation_minus_transformed:
  fixes a b c d :: nat
  fixes x :: "rat mat"
  fixes xv1 :: rat
  assumes v_form: "v = 4 * w - 1"
  assumes w_pos: "0 < w"
  assumes four_sq:
    "k - Λ = a^2 + b^2 + c^2 + d^2"

```

```

shows
  "( $\sum_{i \in \{0..<v\}}.$ 
    ( $\sum_{h \in \{0..<v\}}.$ 
      of_int (N $$ (h,i)) * x $$v (h))^2)
    + of_nat (k -  $\Lambda$ ) * xv1^2
  =
    of_nat  $\Lambda$  *
      ( $\sum_{j \in \{0..<v\}}.$  x $$v (j))^2
    +
      ( $\sum_{h \in \{0..<w\}}.$ 
        y_block_sqsum a b c d
          (brc_extend_x x xv1) h)"
proof -
  have base:
    "( $\sum_{i \in \{0..<v\}}.$ 
      ( $\sum_{h \in \{0..<v\}}.$ 
        of_int (N $$ (h,i)) * x $$v (h))^2)
      + of_nat (k -  $\Lambda$ ) * xv1^2
    =
      of_nat  $\Lambda$  *
        ( $\sum_{j \in \{0..<v\}}.$  x $$v (j))^2
      +
        of_nat (k -  $\Lambda$ ) *
          ( $\sum_{j \in \{0..<v+1\}}.$ 
            (brc_extend_x x xv1 $$v (j))^2)"
  using brc_x_equation_for_extended_vector[of x xv1]
  by simp

  have transformed:
    "of_nat (k -  $\Lambda$ ) *
      ( $\sum_{j \in \{0..<v+1\}}.$ 
        (brc_extend_x x xv1 $$v (j))^2)
    =
      ( $\sum_{h \in \{0..<w\}}.$ 
        y_block_sqsum a b c d
          (brc_extend_x x xv1) h)"
  using brc_extend_x_transformed_blocks[
    OF v_form w_pos four_sq, of x xv1] .

  show ?thesis
    using base transformed
    by simp
qed

end

```

5 Rational elimination of linear squares

This section isolates the algebraic elimination used in both odd-order cases. At one stage a square of a linear form is matched against one diagonal coordinate. The substitution cancels that coordinate, updates the coefficients of all remaining linear forms, and leaves the previously eliminated prefix equal to zero. Iteration reduces the quadratic identity to its final two coordinates while preserving a nontrivial zero. This is the rational-elimination step in the proof of Stinson, Theorem 2.17.

```

definition brc_match_y :: "rat  $\Rightarrow$  rat  $\Rightarrow$  rat" where
  "brc_match_y A R =
    (if A = 1 then -R / 2 else R / (1 - A))"

```

```

lemma brc_match_y_square:
  "(A * brc_match_y A R + R)^2 =
    (brc_match_y A R)^2"
proof (cases "A = 1")
  case True

  then show ?thesis
    unfolding brc_match_y_def
    by (simp add: power2_eq_square algebra_simps)

```

```

next
  case False

```

```

  have nz:
    "1 - A  $\neq$  0"
    using False
    by simp

```

```

  have linear:
    "A * brc_match_y A R + R =
      brc_match_y A R"
  proof -
    have
      "A * (R / (1 - A)) + R =
        (A * R + R * (1 - A)) / (1 - A)"
      using nz
      by (simp add: divide_simps)

```

```

  also have
    "... = R / (1 - A)"
    by (simp add: algebra_simps)

```

```

  finally show ?thesis
    using False
    unfolding brc_match_y_def
    by simp

```

```

qed

show ?thesis
  using linear
  by simp
qed

definition brc_match_coeff :: "rat  $\Rightarrow$  rat  $\Rightarrow$  rat" where
  "brc_match_coeff A B =
    (if A = 1 then -B / 2 else B / (1 - A))"

definition rat_vec_zero ::
  "nat  $\Rightarrow$  rat" where
  "rat_vec_zero i = 0"

definition rat_diagonal_form ::
  "nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat" where
  "rat_diagonal_form n c x =
    ( $\sum_{i \in \{0..<n\}} c\ i * (x\ i)^2$ )"

definition rat_linear_form ::
  "nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat" where
  "rat_linear_form n c y =
    ( $\sum_{j \in \{0..<n\}} c\ j * y\ j$ )"

definition rat_linear_rest ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat" where
  "rat_linear_rest n i c y =
    ( $\sum_{j \in \{0..<n\} - \{i\}} c\ j * y\ j$ )"

definition rat_match_substitution ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$ 
    (nat  $\Rightarrow$  rat)  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "rat_match_substitution n i c y =
    ( $\lambda j.$ 
      if j = i then
        brc_match_y
          (c i)
          (rat_linear_rest n i c y)
      else
        y j)"

lemma rat_linear_form_split:
  assumes i_bound:
    "i < n"
  shows
    "rat_linear_form n c y
    =
    c i * y i +

```

```

      rat_linear_rest n i c y"
proof -
  have i_mem:
    "i ∈ {0.. $n$ }"
    using i_bound
    by simp

  have finite:
    "finite {0.. $n$ }"
    by simp

  have split:
    " $(\sum_{j \in \{0.. $n$ \}} c_j * y_j)$ "
    =
    " $c_i * y_i + (\sum_{j \in \{0.. $n$ \} - \{i\}} c_j * y_j)$ "
    using sum.remove[OF finite i_mem,
      of " $\lambda j. c_j * y_j$ "]
    by simp

  show ?thesis
    unfolding rat_linear_form_def
      rat_linear_rest_def
    using split
    by simp
qed

lemma rat_linear_rest_match_substitution:
  "rat_linear_rest n i c
    (rat_match_substitution n i c y)
  =
  rat_linear_rest n i c y"
proof -
  show ?thesis
    unfolding rat_linear_rest_def
      rat_match_substitution_def
    by (intro sum.cong) auto
qed

lemma rat_match_substitution_at:
  "rat_match_substitution n i c y i
  =
  brc_match_y
    (c i)
    (rat_linear_rest n i c y)"
  unfolding rat_match_substitution_def
  by simp

lemma rat_linear_form_match_square:

```

```

assumes i_bound:
  "i < n"
shows
  "(rat_linear_form n c
    (rat_match_substitution n i c y))^2
  =
  (rat_match_substitution n i c y i)^2"
proof -
  let ?R =
    "rat_linear_rest n i c y"

  let ?z =
    "rat_match_substitution n i c y"

  have rest:
    "rat_linear_rest n i c ?z = ?R"
  using rat_linear_rest_match_substitution[
    of n i c y]
  .

  have coordinate:
    "?z i = brc_match_y (c i) ?R"
  using rat_match_substitution_at[
    of n i c y]
  .

  have split:
    "rat_linear_form n c ?z
    =
    c i * ?z i +
    rat_linear_rest n i c ?z"
  using rat_linear_form_split[
    OF i_bound,
    of c ?z]
  .

  have form:
    "rat_linear_form n c ?z
    =
    c i * brc_match_y (c i) ?R + ?R"
  using split coordinate rest
  by simp

  have matched:
    "(c i * brc_match_y (c i) ?R + ?R)^2
    =
    (brc_match_y (c i) ?R)^2"
  using brc_match_y_square[
    of "c i" ?R]

```

```

.

show ?thesis
  using form coordinate matched
  by simp
qed

lemma rat_diagonal_form_split:
  fixes c :: "nat  $\Rightarrow$  rat"
  assumes i_bound:
    "i < n"
  shows
    "rat_diagonal_form n c x
     =
     c i * (x i)^2 +
     ( $\sum_{j \in \{0..<n\} - \{i\}} c j * (x j)^2$ )"
proof -
  have i_mem:
    "i  $\in$  {0.. $<n$ }"
  using i_bound
  by simp

  have finite:
    "finite {0.. $<n$ }"
  by simp

  have split:
    " ( $\sum_{j \in \{0..<n\}} c j * (x j)^2$ )
      =
      c i * (x i)^2 +
      ( $\sum_{j \in \{0..<n\} - \{i\}} c j * (x j)^2$ ) "
  using sum.remove[OF finite i_mem,
    of "\j. c j * (x j)^2"]
  by simp

  show ?thesis
    unfolding rat_diagonal_form_def
    using split
    by simp
qed

lemma rat_diagonal_rest_match_substitution:
  fixes d :: "nat  $\Rightarrow$  rat"
  shows
    " ( $\sum_{j \in \{0..<n\} - \{i\}} d j *
      (rat_match_substitution n i c y j)^2$ ) "

```

```

    =
    (∑ j ∈ {0..<n} - {i}.
      d j * (y j)^2)"
proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
    unfolding rat_match_substitution_def
    by auto
qed

lemma rat_match_substitution_cancels_coordinate:
  fixes c d :: "nat ⇒ rat"
  assumes i_bound:
    "i < n"
  assumes unit_coeff:
    "d i = 1"
  shows
    "rat_diagonal_form n d
      (rat_match_substitution n i c y)
    -
    (rat_linear_form n c
      (rat_match_substitution n i c y))^2
    =
    (∑ j ∈ {0..<n} - {i}.
      d j * (y j)^2)"
proof -
  let ?z =
    "rat_match_substitution n i c y"

  have diagonal:
    "rat_diagonal_form n d ?z
    =
    d i * (?z i)^2 +
    (∑ j ∈ {0..<n} - {i}.
      d j * (?z j)^2)"
  using rat_diagonal_form_split[
    OF i_bound, of d ?z]
  .

  have square:
    "(rat_linear_form n c ?z)^2
    =
    (?z i)^2"
  using rat_linear_form_match_square[
    OF i_bound, of c y]
  .

  have rest:

```

```

      "(\sum j \in \{0..<n\} - \{i\}.
        d j * (?z j)^2)
    =
      "(\sum j \in \{0..<n\} - \{i\}.
        d j * (y j)^2)"
    using rat_diagonal_rest_match_substitution[
      where n = n
        and i = i
        and d = d
        and c = c
        and y = y]
    .

  show ?thesis
    using diagonal_square_rest_unit_coeff
    by simp
qed

definition rat_zero_coordinate ::
  "nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "rat_zero_coordinate i y =
    (\lambda j. if j = i then 0 else y j)"

lemma rat_zero_coordinate_at:
  "rat_zero_coordinate i y i = 0"
  unfolding rat_zero_coordinate_def
  by simp

lemma rat_zero_coordinate_other:
  assumes ji:
    "j  $\neq$  i"
  shows
    "rat_zero_coordinate i y j = y j"
  using ji
  unfolding rat_zero_coordinate_def
  by simp

lemma rat_diagonal_form_zero_coordinate:
  fixes d :: "nat  $\Rightarrow$  rat"
  assumes i_bound:
    "i < n"
  shows
    "rat_diagonal_form n d
      (rat_zero_coordinate i y)
    =
      "(\sum j \in \{0..<n\} - \{i\}.
        d j * (y j)^2)"
  proof -
    have split:

```

```

"rat_diagonal_form n d
  (rat_zero_coordinate i y)
=
d i * (rat_zero_coordinate i y i)^2 +
(∑ j∈{0..<n} - {i}.
  d j *
    (rat_zero_coordinate i y j)^2)"
using rat_diagonal_form_split[
  where n = n
    and i = i
    and c = d
    and x = "rat_zero_coordinate i y",
  OF i_bound]
.

have at_zero:
  "rat_zero_coordinate i y i = 0"
  using rat_zero_coordinate_at .

have rest:
  "(∑ j∈{0..<n} - {i}.
    d j * (rat_zero_coordinate i y j)^2)
  =
  (∑ j∈{0..<n} - {i}.
    d j * (y j)^2)"
proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
  using rat_zero_coordinate_other
  by auto
qed

show ?thesis
  using split at_zero rest
  by simp
qed

lemma brc_match_y_linear:
  "brc_match_y A (B * y)
  =
  brc_match_coeff A B * y"
unfolding brc_match_y_def brc_match_coeff_def
by (cases "A = 1"; simp add: algebra_simps)

lemma brc_match_y_add:
  "brc_match_y A (R + S)
  =
  brc_match_y A R + brc_match_y A S"

```

```

proof (cases "A = 1")
  case True

  show ?thesis
    unfolding brc_match_y_def
    using True
    by (simp; linarith)
next
  case False

  have nz:
    "1 - A  $\neq$  0"
    using False
    by simp

  show ?thesis
    unfolding brc_match_y_def
    using False nz
    by (simp add: add_divide_distrib)
qed

lemma brc_match_y_sum:
  assumes finite:
    "finite S"
  shows
    "brc_match_y A ( $\sum_{j \in S}. f j$ )
    =
    ( $\sum_{j \in S}. \text{brc\_match\_y } A (f j)$ )"
  using finite
proof (induction S rule: finite_induct)
  case empty

  show ?case
    unfolding brc_match_y_def
    by simp
next
  case (insert j S)

  have add:
    "brc_match_y A
    (f j + ( $\sum_{k \in S}. f k$ ))
    =
    brc_match_y A (f j) +
    brc_match_y A ( $\sum_{k \in S}. f k$ )"
  using brc_match_y_add[
    where A = A
    and R = "f j"
    and S = " $\sum_{k \in S}. f k$ "]
  .

```

```

show ?case
  using insert add
  by simp
qed

lemma rat_match_substitution_at_linear:
  "rat_match_substitution n i c y i
  =
  ( $\sum_{j \in \{0..<n\} - \{i\}}.$ 
    brc_match_coeff (c i) (c j) * y j)"
proof -
  have at:
    "rat_match_substitution n i c y i
    =
    brc_match_y
      (c i)
      (rat_linear_rest n i c y)"
  using rat_match_substitution_at[
    where n = n
    and i = i
    and c = c
    and y = y]
  .

  have expanded:
    "brc_match_y
      (c i)
      (rat_linear_rest n i c y)
    =
    ( $\sum_{j \in \{0..<n\} - \{i\}}.$ 
      brc_match_y
        (c i)
        (c j * y j))"
  proof -
    show ?thesis
      unfolding rat_linear_rest_def
      using brc_match_y_sum[
        where A = "c i"
        and S = "{0..<n} - {i}"
        and f = "\j. c j * y j"]
      by simp
  qed

  have linear:
    "( $\sum_{j \in \{0..<n\} - \{i\}}.$ 
      brc_match_y
        (c i)
        (c j * y j))"

```

```

=
  (∑ j ∈ {0..<n} - {i}.
    brc_match_coeff (c i) (c j) * y j)"
proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
    using brc_match_y_linear
    by simp
qed

show ?thesis
  using at expanded linear
  by simp
qed

definition rat_match_pullback_coeff ::
  "nat ⇒ nat ⇒
  (nat ⇒ rat) ⇒
  (nat ⇒ rat) ⇒
  nat ⇒ rat" where
  "rat_match_pullback_coeff n i c e =
    (λj.
      if j = i then
        0
      else
        e j +
        e i * brc_match_coeff (c i) (c j))"

lemma rat_linear_form_match_substitution:
  fixes c e :: "nat ⇒ rat"
  assumes i_bound:
    "i < n"
  shows
    "rat_linear_form n e
      (rat_match_substitution n i c y)
    =
      rat_linear_form n
        (rat_match_pullback_coeff n i c e)
        y"
proof -
  let ?z =
    "rat_match_substitution n i c y"

  let ?p =
    "rat_match_pullback_coeff n i c e"

  have split:
    "rat_linear_form n e ?z

```

```

=
e i * ?z i +
rat_linear_rest n i e ?z"
using rat_linear_form_split[
  where n = n
    and i = i
    and c = e
    and y = ?z,
  OF i_bound]
.

have rest:
  "rat_linear_rest n i e ?z
  =
  ( $\sum_{j \in \{0..<n\} - \{i\}} e j * y j$ )"
proof -
  show ?thesis
    unfolding rat_linear_rest_def
    apply (rule sum.cong)
    apply (rule refl)
    unfolding rat_match_substitution_def
    by auto
qed

have coordinate:
  "?z i
  =
  ( $\sum_{j \in \{0..<n\} - \{i\}} brc\_match\_coeff (c i) (c j) * y j$ )"
using rat_match_substitution_at_linear[
  where n = n
    and i = i
    and c = c
    and y = y]
.

have left:
  "rat_linear_form n e ?z
  =
  ( $\sum_{j \in \{0..<n\} - \{i\}} (e j + e i * brc\_match\_coeff (c i) (c j)) * y j$ )"
using split rest coordinate
by (simp add: sum_distrib_left sum.distrib algebra_simps)

have p_at:
  "?p i = 0"
unfolding rat_match_pullback_coeff_def

```

```

    by simp

have right:
  "rat_linear_form n ?p y
  =
  ( $\sum_{j \in \{0..<n\} - \{i\}} (e_j + e_i * brc\_match\_coeff (c\ i) (c\ j)) * y\ j)$ "
proof -
  have form_split:
    "rat_linear_form n ?p y
    =
    ?p i * y i +
    rat_linear_rest n i ?p y"
  using rat_linear_form_split[
    where n = n
    and i = i
    and c = ?p
    and y = y,
    OF i_bound]
  .

  have rest_expanded:
    "rat_linear_rest n i ?p y
    =
    ( $\sum_{j \in \{0..<n\} - \{i\}} (e_j + e_i * brc\_match\_coeff (c\ i) (c\ j)) * y\ j)$ "
  unfolding rat_linear_rest_def
    rat_match_pullback_coeff_def
  by (intro sum.cong) auto

  show ?thesis
    using form_split rest_expanded p_at
    by simp
qed

show ?thesis
  using left right
  by simp
qed

definition rat_elimination_update ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$ 
  (nat  $\Rightarrow$  nat  $\Rightarrow$  rat)  $\Rightarrow$ 
  nat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "rat_elimination_update n q C =

```

```

      (λr.
        rat_match_pullback_coeff
          n q (C q) (C r))"

primrec rat_elimination_coeffs ::
  "nat ⇒ nat ⇒
    (nat ⇒ nat ⇒ rat) ⇒
    nat ⇒ nat ⇒ rat" where
  "rat_elimination_coeffs n 0 C = C"
| "rat_elimination_coeffs n (Suc q) C =
  rat_elimination_update
    n q
    (rat_elimination_coeffs n q C)"

lemma brc_match_coeff_zero:
  "brc_match_coeff A 0 = 0"
  unfolding brc_match_coeff_def
  by simp

lemma rat_elimination_update_preserves_zero:
  assumes r_zero:
    "C r j = 0"
  assumes pivot_zero:
    "C q j = 0"
  shows
    "rat_elimination_update n q C r j = 0"
proof (cases "j = q")
  case True

  show ?thesis
  using True
  unfolding rat_elimination_update_def
    rat_match_pullback_coeff_def
  by simp

next
  case False

  have matched_zero:
    "brc_match_coeff (C q q) (C q j) = 0"
  using pivot_zero brc_match_coeff_zero
  by simp

  show ?thesis
  using False r_zero matched_zero
  unfolding rat_elimination_update_def
    rat_match_pullback_coeff_def
  by simp
qed

```

47

```

have pivot_zero:
  "rat_elimination_coeffs n q C q j = 0"
  using previous
  by blast

have updated_zero:
  "rat_elimination_update
    n q
    (rat_elimination_coeffs n q C)
    r j
  =
  0"
  using rat_elimination_update_preserves_zero[
    where n = n
      and q = q
      and C = "rat_elimination_coeffs n q C"
      and r = r
      and j = j,
    OF r_zero pivot_zero]
  .

show ?thesis
  using updated_zero
  by simp
qed
qed
qed

show ?thesis
  using all_r
  by blast
qed

definition rat_zero_prefix ::
  "nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "rat_zero_prefix q y =
    ( $\lambda$ j. if j < q then 0 else y j)"

lemma rat_zero_prefix_below:
  assumes j_lt:
    "j < q"
  shows
    "rat_zero_prefix q y j = 0"
  using j_lt
  unfolding rat_zero_prefix_def
  by simp

lemma rat_zero_prefix_from:

```

```

    assumes q_le:
      "q ≤ j"
    shows
      "rat_zero_prefix q y j = y j"
    using q_le
    unfolding rat_zero_prefix_def
    by simp

lemma rat_linear_form_zero_prefix:
  fixes c :: "nat ⇒ rat"
  assumes coefficients_zero:
    "∧j. j < q ⇒ c j = 0"
  shows
    "rat_linear_form n c
      (rat_zero_prefix q y)
    =
      rat_linear_form n c y"
proof -
  show ?thesis
    unfolding rat_linear_form_def
  proof (rule sum.cong)
    show "{0..

```

```

      have coordinate:
        "rat_zero_prefix q y j = y j"
        using rat_zero_prefix_from[OF q_le] .

      show ?thesis
        using coordinate
        by simp
    qed
  qed
qed

lemma rat_match_substitution_preserves_zero_prefix:
  assumes q_bound:
    "q < n"
  shows
    "rat_zero_prefix q
      (rat_match_substitution
        n q c (rat_zero_prefix q y))
    =
      rat_match_substitution
        n q c (rat_zero_prefix q y)"
proof
  fix j

  show
    "rat_zero_prefix q
      (rat_match_substitution
        n q c (rat_zero_prefix q y)) j
    =
      rat_match_substitution
        n q c (rat_zero_prefix q y) j"
proof (cases "j < q")
  case True

  have jq:
    "j ≠ q"
    using True
    by simp

  have input_zero:
    "rat_zero_prefix q y j = 0"
    using rat_zero_prefix_below[OF True] .

  have substituted_zero:
    "rat_match_substitution
      n q c (rat_zero_prefix q y) j = 0"
    using jq input_zero
    unfolding rat_match_substitution_def

```

```

      by simp

    show ?thesis
      using True substituted_zero
      unfolding rat_zero_prefix_def
      by simp
  next
    case False

    have q_le:
      "q ≤ j"
      using False
      by simp

    show ?thesis
      using q_le
      unfolding rat_zero_prefix_def
      by simp
  qed
qed

lemma rat_zero_coordinate_zero_prefix_Suc:
  "rat_zero_coordinate q
   (rat_zero_prefix (Suc q) y)
  =
   rat_zero_prefix (Suc q) y"
proof
  fix j

  show
    "rat_zero_coordinate q
     (rat_zero_prefix (Suc q) y) j
    =
     rat_zero_prefix (Suc q) y j"
  proof (cases "j = q")
    case True

    have q_lt:
      "q < Suc q"
      by simp

    have prefix_zero:
      "rat_zero_prefix (Suc q) y q = 0"
      using rat_zero_prefix_below[OF q_lt] .

    show ?thesis
      using True prefix_zero
      unfolding rat_zero_coordinate_def
      by simp
  end
end

```

```

next
  case False

  show ?thesis
    using False
    unfolding rat_zero_coordinate_def
    by simp
qed
qed

lemma rat_zero_prefix_nested:
  assumes q_le:
    " $q \leq s$ "
  shows
    "rat_zero_prefix q
      (rat_zero_prefix s y)
    =
      rat_zero_prefix s y"
proof
  fix j

  show
    "rat_zero_prefix q
      (rat_zero_prefix s y) j
    =
      rat_zero_prefix s y j"
proof (cases " $j < q$ ")
  case True

  have j_lt_s:
    " $j < s$ "
    using True q_le
    by simp

  have inner_zero:
    "rat_zero_prefix s y j = 0"
    using rat_zero_prefix_below[OF j_lt_s] .

  show ?thesis
    using True inner_zero
    unfolding rat_zero_prefix_def
    by simp
next
  case False

  have q_le_j:
    " $q \leq j$ "
    using False
    by simp

```

```

    show ?thesis
      using q_le_j
      unfolding rat_zero_prefix_def
      by simp
  qed
qed

definition rat_weighted_linear_squares_from ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
  (nat  $\Rightarrow$  rat)  $\Rightarrow$ 
  (nat  $\Rightarrow$  nat  $\Rightarrow$  rat)  $\Rightarrow$ 
  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat" where
  "rat_weighted_linear_squares_from n q m W C y =
    ( $\sum_{r \in \{q..<m\}}$ .
      W r * (rat_linear_form n (C r) y)^2)"

lemma rat_weighted_linear_squares_from_split:
  assumes q_lt:
    "q < m"
  shows
    "rat_weighted_linear_squares_from n q m W C y
     =
     W q * (rat_linear_form n (C q) y)^2 +
     rat_weighted_linear_squares_from
       n (Suc q) m W C y"
proof -
  have interval:
    "{q..<m} = insert q {Suc q..<m}"
  using q_lt
  by auto

  have q_not_mem:
    "q  $\notin$  {Suc q..<m}"
  by simp

  show ?thesis
    unfolding rat_weighted_linear_squares_from_def
    using interval q_not_mem
    by simp
qed

lemma rat_weighted_linear_squares_from_match:
  assumes i_bound:
    "i < n"
  shows
    "rat_weighted_linear_squares_from n q m W C
     (rat_match_substitution n i c y)
     =

```

```

    rat_weighted_linear_squares_from n q m W
      (λr.
        rat_match_pullback_coeff
          n i c (C r))
      y"
proof -
  show ?thesis
    unfolding rat_weighted_linear_squares_from_def
  proof (rule sum.cong)
    show "{q.. $m\} = \{q.. $m\}"
      by simp
  next
    fix r
    assume r_mem:
      "r ∈ {q.. $m\}"

    have form:
      "rat_linear_form n (C r)
        (rat_match_substitution n i c y)
        =
        rat_linear_form n
          (rat_match_pullback_coeff
            n i c (C r))
          y"
    using rat_linear_form_match_substitution[
      where n = n
        and i = i
        and c = c
        and e = "C r"
        and y = y,
      OF i_bound]
    .

    show
      "W r *
        (rat_linear_form n (C r)
          (rat_match_substitution n i c y))^2
        =
        W r *
          (rat_linear_form n
            (rat_match_pullback_coeff
              n i c (C r))
            y)^2"
    using form
    by simp
  qed
qed

lemma rat_weighted_sum_elimination_zero_prefix:$$$ 
```

```

"rat_weighted_linear_squares_from
  n q m W
  (rat_elimination_coeffs n q C)
  (rat_zero_prefix q y)
=
rat_weighted_linear_squares_from
  n q m W
  (rat_elimination_coeffs n q C)
  y"
proof -
  show ?thesis
    unfolding rat_weighted_linear_squares_from_def
  proof (rule sum.cong)
    show "{q.. $m\} = \{q.. $m\}"
      by simp
  next
    fix r
    assume r_mem:
      "r  $\in \{q.. $m\}"

    have form:
      "rat_linear_form n
        (rat_elimination_coeffs n q C r)
        (rat_zero_prefix q y)
      =
      rat_linear_form n
        (rat_elimination_coeffs n q C r)
        y"
    proof (rule rat_linear_form_zero_prefix)
      fix j
      assume j_lt:
        "j < q"

      show
        "rat_elimination_coeffs n q C r j = 0"
        using rat_elimination_coeffs_previous_zero[
          where n = n
            and q = q
            and C = C
            and r = r
            and j = j,
          OF j_lt]
        .
    qed

    show
      "W r *
        (rat_linear_form n
          (rat_elimination_coeffs n q C r)$$$$ 
```

```

      (rat_zero_prefix q y))^2
    =
    W r *
    (rat_linear_form n
      (rat_elimination_coeffs n q C r)
      y)^2"
  using form
  by simp
qed
qed

```

This is the single-step invariant for rational elimination. Assuming that the first q coordinates have already been eliminated, the matching substitution cancels coordinate q , preserves the earlier zero prefix, and rewrites the weighted sum of squared linear forms as the same diagonal quadratic form. Iterating this step leaves only the terminal linear forms.

```

lemma rat_weighted_elimination_stage_step:
  fixes W :: "nat ⇒ rat"
  fixes C :: "nat ⇒ nat ⇒ rat"
  fixes d :: "nat ⇒ rat"
  assumes q_lt_m:
    "q < m"
  assumes q_lt_n:
    "q < n"
  assumes weight_one:
    "W q = 1"
  assumes unit:
    "d q = 1"
  assumes stage:
    "∧x.
      rat_weighted_linear_squares_from
        n q m W
        (rat_elimination_coeffs n q C) x
      =
      rat_diagonal_form n d
        (rat_zero_prefix q x)"
  shows
    "∧y.
      rat_weighted_linear_squares_from
        n (Suc q) m W
        (rat_elimination_coeffs n (Suc q) C) y
      =
      rat_diagonal_form n d
        (rat_zero_prefix (Suc q) y)"
proof -
  fix y

  let ?D =
    "rat_elimination_coeffs n q C"

```

```

let ?p =
  "?D q"

let ?u =
  "rat_zero_prefix (Suc q) y"

let ?z =
  "rat_match_substitution n q ?p ?u"

have u_prefix:
  "rat_zero_prefix q ?u = ?u"
  using rat_zero_prefix_nested[
    where q = q
      and s = "Suc q"
      and y = y]
  by simp

have z_prefix:
  "rat_zero_prefix q ?z = ?z"
proof -
  have preserved:
    "rat_zero_prefix q
      (rat_match_substitution
        n q ?p (rat_zero_prefix q ?u))
    =
    rat_match_substitution
      n q ?p (rat_zero_prefix q ?u)"
  using rat_match_substitution_preserves_zero_prefix[
    where n = n
      and q = q
      and c = ?p
      and y = ?u,
    OF q_lt_n]
  .

  show ?thesis
    using preserved u_prefix
    by simp
qed

have stage_z:
  "rat_weighted_linear_squares_from
    n q m W ?D ?z
  =
  rat_diagonal_form n d ?z"
  using stage[of ?z] z_prefix
  by simp

```

```

have split:
  "rat_weighted_linear_squares_from
    n q m W ?D ?z
  =
  (rat_linear_form n ?p ?z)^2 +
  rat_weighted_linear_squares_from
    n (Suc q) m W ?D ?z"
proof -
  have raw:
    "rat_weighted_linear_squares_from
      n q m W ?D ?z
    =
    W q * (rat_linear_form n (?D q) ?z)^2 +
    rat_weighted_linear_squares_from
      n (Suc q) m W ?D ?z"
  using rat_weighted_linear_squares_from_split[
    where n = n
      and q = q
      and m = m
      and W = W
      and C = ?D
      and y = ?z,
    OF q_lt_m]
  .

  show ?thesis
    using raw weight_one
    by simp
qed

have remaining:
  "rat_weighted_linear_squares_from
    n (Suc q) m W ?D ?z
  =
  rat_diagonal_form n d ?z -
  (rat_linear_form n ?p ?z)^2"
  using stage_z split
  by linarith

have cancellation:
  "rat_diagonal_form n d ?z -
    (rat_linear_form n ?p ?z)^2
  =
  (∑ j∈{0..

```

```

      and d = d
      and y = ?u,
    OF q_lt_n unit]
.

have diagonal_u:
  "rat_diagonal_form n d ?u
  =
  ( $\sum_{j \in \{0..<n\} - \{q\}} d\ j * (?u\ j)^2$ )"
proof -
  have zeroed:
    "rat_zero_coordinate q ?u = ?u"
    using rat_zero_coordinate_zero_prefix_Suc[
      where q = q and y = y]
  .

  have diagonal:
    "rat_diagonal_form n d
    (rat_zero_coordinate q ?u)
    =
    ( $\sum_{j \in \{0..<n\} - \{q\}} d\ j * (?u\ j)^2$ )"
    using rat_diagonal_form_zero_coordinate[
      where n = n
      and i = q
      and d = d
      and y = ?u,
      OF q_lt_n]
  .

  show ?thesis
    using zeroed diagonal
    by simp
qed

have transformed:
  "rat_weighted_linear_squares_from
  n (Suc q) m W ?D ?z
  =
  rat_weighted_linear_squares_from
  n (Suc q) m W
  (rat_elimination_coeffs n (Suc q) C)
  ?u"
  using rat_weighted_linear_squares_from_match[
    where n = n
    and q = "Suc q"
    and m = m
    and W = W

```

```

    and i = q
    and c = ?p
    and C = ?D
    and y = ?u,
  OF q_lt_n]
by (simp add: rat_elimination_update_def)

have at_u:
  "rat_weighted_linear_squares_from
    n (Suc q) m W
    (rat_elimination_coeffs n (Suc q) C)
    ?u
  =
  rat_diagonal_form n d ?u"
using remaining_cancellation_diagonal_u_transformed
by simp

have remove_prefix:
  "rat_weighted_linear_squares_from
    n (Suc q) m W
    (rat_elimination_coeffs n (Suc q) C)
    ?u
  =
  rat_weighted_linear_squares_from
    n (Suc q) m W
    (rat_elimination_coeffs n (Suc q) C)
    y"
using rat_weighted_sum_elimination_zero_prefix[
  where n = n
  and q = "Suc q"
  and m = m
  and W = W
  and C = C
  and y = y]
.

show
  "rat_weighted_linear_squares_from
    n (Suc q) m W
    (rat_elimination_coeffs n (Suc q) C) y
  =
  rat_diagonal_form n d
    (rat_zero_prefix (Suc q) y)"
using at_u remove_prefix
by simp
qed

```

Induction on the elimination stage now iterates the preceding one-step invariant. After q stages, the first q coordinates vanish, the coefficient arrays record the accumulated substitutions, and the weighted quadratic identity

remains valid.

```

lemma rat_weighted_elimination_iterate:
  fixes W :: "nat  $\Rightarrow$  rat"
  fixes C :: "nat  $\Rightarrow$  nat  $\Rightarrow$  rat"
  fixes d :: "nat  $\Rightarrow$  rat"
  assumes Q_le_m:
    " $Q \leq m$ "
  assumes Q_le_n:
    " $Q \leq n$ "
  assumes weights:
    " $\bigwedge q. q < Q \implies W\ q = 1$ "
  assumes units:
    " $\bigwedge q. q < Q \implies d\ q = 1$ "
  assumes initial:
    " $\bigwedge x.$ 
      rat_weighted_linear_squares_from
        n 0 m W C x
      =
      rat_diagonal_form n d x"
  shows
    " $\bigwedge y.$ 
      rat_weighted_linear_squares_from
        n Q m W
        (rat_elimination_coeffs n Q C) y
      =
      rat_diagonal_form n d
        (rat_zero_prefix Q y)"
  using Q_le_m Q_le_n weights units
proof (induction Q)
  case 0

  fix y

  have initial_y:
    "rat_weighted_linear_squares_from
      n 0 m W C y
    =
    rat_diagonal_form n d y"
  using initial[of y] .

  show
    "rat_weighted_linear_squares_from
      n 0 m W
      (rat_elimination_coeffs n 0 C) y
    =
    rat_diagonal_form n d
      (rat_zero_prefix 0 y)"
  using initial_y
  unfolding rat_zero_prefix_def

```

```

      by simp
next
  case (Suc Q)

  have Q_le_m:
    "Q ≤ m"
    using Suc.prem1(1)
    by simp

  have Q_le_n:
    "Q ≤ n"
    using Suc.prem1(2)
    by simp

  have weights_Q:
    "∧q. q < Q ⇒ W q = 1"
    using Suc.prem1(3)
    by simp

  have units_Q:
    "∧q. q < Q ⇒ d q = 1"
    using Suc.prem1(4)
    by simp

  have stage_Q:
    "∧x.
      rat_weighted_linear_squares_from
        n Q m W
      (rat_elimination_coeffs n Q C) x
    =
      rat_diagonal_form n d
      (rat_zero_prefix Q x)"
    using Suc.IH[
      OF Q_le_m Q_le_n weights_Q units_Q]
    .

  have Q_lt_m:
    "Q < m"
    using Suc.prem1(1)
    by simp

  have Q_lt_n:
    "Q < n"
    using Suc.prem1(2)
    by simp

  have weight_Q:
    "W Q = 1"
    using Suc.prem1(3)

```

```

    by simp

have unit_Q:
  "d Q = 1"
  using Suc.prem(4)
  by simp

show ?case
  using rat_weighted_elimination_stage_step[
    where n = n
    and q = Q
    and m = m
    and W = W
    and C = C
    and d = d,
    OF Q_lt_m Q_lt_n weight_Q unit_Q stage_Q]
  .
qed

lemma rat_weighted_terminal_two:
  assumes n_pos:
    "0 < n"
  shows
    "rat_weighted_linear_squares_from
      n (n - 1) (n + 1) W C y
    =
      W (n - 1) *
      (rat_linear_form n (C (n - 1)) y)^2
    +
      W n *
      (rat_linear_form n (C n) y)^2"
  proof -
    obtain t where n_eq:
      "n = Suc t"
    using n_pos
    by (cases n) auto

    show ?thesis
      unfolding rat_weighted_linear_squares_from_def
      using n_eq
      by simp
  qed

lemma rat_diagonal_form_zero_prefix_last:
  assumes n_pos:
    "0 < n"
  shows
    "rat_diagonal_form n d
      (rat_zero_prefix (n - 1) y)"

```

```

    =
    d (n - 1) * (y (n - 1))^2"
proof -
  obtain t where n_eq:
    "n = Suc t"
  using n_pos
  by (cases n) auto

  show ?thesis
    unfolding rat_diagonal_form_def
      rat_zero_prefix_def
    using n_eq
    by simp
qed

```

```

definition rat_unit_coordinate ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "rat_unit_coordinate k =
    ( $\lambda$ j. if j = k then 1 else 0)"

```

The following terminal elimination theorems depend only on the rational forms introduced above. They make no use of a block design and therefore belong outside *ordered_sym_bibd*.

```

lemma rat_weighted_elimination_terminal_general:
  fixes C :: "nat  $\Rightarrow$  nat  $\Rightarrow$  rat"
  fixes d W :: "nat  $\Rightarrow$  rat"
  fixes alpha beta delta :: rat
  assumes n_pos:
    "0 < n"
  assumes elimination_weights:
    " $\bigwedge$ q. q < n - 1  $\implies$  W q = 1"
  assumes first_terminal_weight:
    "W (n - 1) = alpha"
  assumes second_terminal_weight:
    "W n = beta"
  assumes unit_diagonal:
    " $\bigwedge$ q. q < n - 1  $\implies$  d q = 1"
  assumes last_diagonal:
    "d (n - 1) = delta"
  assumes initial:
    " $\bigwedge$ x.
      rat_weighted_linear_squares_from
        n 0 (n + 1) W C x
    =
      rat_diagonal_form n d x"
  shows
    " $\bigwedge$ y.
      alpha *
      (rat_linear_form n

```

```

      (rat_elimination_coeffs
        n (n - 1) C (n - 1))
      y)^2
    +
    beta *
    (rat_linear_form n
      (rat_elimination_coeffs
        n (n - 1) C n)
      y)^2
    =
    delta * (y (n - 1))^2"
proof -
  fix y

  have iterated:
    "rat_weighted_linear_squares_from
      n (n - 1) (n + 1) W
      (rat_elimination_coeffs n (n - 1) C) y
    =
    rat_diagonal_form n d
      (rat_zero_prefix (n - 1) y)"
  using rat_weighted_elimination_iterate[
    where n = n
      and Q = "n - 1"
      and m = "n + 1"
      and W = W
      and C = C
      and d = d,
    OF _ _ elimination_weights
      unit_diagonal initial]
  by simp

  have left:
    "rat_weighted_linear_squares_from
      n (n - 1) (n + 1) W
      (rat_elimination_coeffs n (n - 1) C) y
    =
    alpha *
    (rat_linear_form n
      (rat_elimination_coeffs
        n (n - 1) C (n - 1))
      y)^2
    +
    beta *
    (rat_linear_form n
      (rat_elimination_coeffs
        n (n - 1) C n)
      y)^2"
  using rat_weighted_terminal_two[

```

```

    where n = n
      and W = W
      and C = "rat_elimination_coeffs n (n - 1) C"
      and y = y,
    OF n_pos]
  first_terminal_weight second_terminal_weight
by simp

have right:
  "rat_diagonal_form n d
    (rat_zero_prefix (n - 1) y)
  =
  delta * (y (n - 1))^2"
using rat_diagonal_form_zero_prefix_last[
  where n = n and d = d and y = y,
  OF n_pos]
  last_diagonal
by simp

show
  "alpha *
    (rat_linear_form n
      (rat_elimination_coeffs
        n (n - 1) C (n - 1))
      y)^2
  +
  beta *
    (rat_linear_form n
      (rat_elimination_coeffs
        n (n - 1) C n)
      y)^2
  =
  delta * (y (n - 1))^2"
using iterated left right
by simp
qed

```

The elimination leaves one diagonal coordinate free. We set this coordinate to one and evaluate the two surviving linear forms. The terminal identity then supplies a rational solution of the required quadratic equation. This solution is nontrivial because one of its coordinates is one. Multiplication by a common nonzero denominator gives a nontrivial integer solution.

```

lemma rat_weighted_elimination_terminal_solution_general:
  fixes C :: "nat ⇒ nat ⇒ rat"
  fixes d W :: "nat ⇒ rat"
  fixes alpha beta :: rat
  assumes n_pos:
    "0 < n"
  assumes elimination_weights:

```

```

    "∧q. q < n - 1 ⇒ W q = 1"
  assumes first_terminal_weight:
    "W (n - 1) = alpha"
  assumes second_terminal_weight:
    "W n = beta"
  assumes unit_diagonal:
    "∧q. q < n - 1 ⇒ d q = 1"
  assumes last_diagonal:
    "d (n - 1) = 1"
  assumes initial:
    "∧x.
      rat_weighted_linear_squares_from
        n 0 (n + 1) W C x
      =
      rat_diagonal_form n d x"
  shows
    "∃x y z :: rat.
      (x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0) ∧
      x^2 = alpha * y^2 + beta * z^2"
proof -
  let ?u = "rat_unit_coordinate (n - 1)"
  let ?y = "rat_linear_form n (rat_elimination_coeffs n (n - 1) C (n -
1)) ?u"
  let ?z = "rat_linear_form n (rat_elimination_coeffs n (n - 1) C n) ?u"
  have equation: "(1::rat)^2 = alpha * ?y^2 + beta * ?z^2"
    using rat_weighted_elimination_terminal_general[
      where n = n and C = C and d = d and W = W
      and alpha = alpha and beta = beta and delta = 1,
      OF n_pos elimination_weights first_terminal_weight second_terminal_weight
      unit_diagonal last_diagonal initial, of ?u]
    by (simp add: rat_unit_coordinate_def)
  show ?thesis
    apply (rule exI[of _ 1])
    apply (rule exI[of _ ?y])
    apply (rule exI[of _ ?z])
    using equation by simp
qed

lemma rat_weighted_elimination_terminal:
  fixes C :: "nat ⇒ nat ⇒ rat"
  fixes d W :: "nat ⇒ rat"
  fixes lam delta :: rat
  assumes n_pos:
    "0 < n"
  assumes form_weights:
    "∧q. q < n ⇒ W q = 1"
  assumes carried_weight:
    "W n = -lam"
  assumes unit_diagonal:

```

```

    "∧q. q < n - 1 ⇒ d q = 1"
  assumes last_diagonal:
    "d (n - 1) = delta"
  assumes initial:
    "∧x.
      rat_weighted_linear_squares_from
        n 0 (n + 1) W C x
      =
      rat_diagonal_form n d x"
  shows
    "∧y.
      (rat_linear_form n
        (rat_elimination_coeffs
          n (n - 1) C (n - 1))
        y)^2
      -
      lam *
      (rat_linear_form n
        (rat_elimination_coeffs
          n (n - 1) C n)
        y)^2
      =
      delta * (y (n - 1))^2"
  proof -
    have weights: "∧q. q < n - 1 ⇒ W q = 1"
      using form_weights by simp
    have last_weight: "W (n - 1) = 1"
      using form_weights n_pos by simp
    show "∧y.
      (rat_linear_form n (rat_elimination_coeffs n (n - 1) C (n - 1)) y)^2
      - lam * (rat_linear_form n (rat_elimination_coeffs n (n - 1) C n)
        y)^2
      = delta * (y (n - 1))^2"
      using rat_weighted_elimination_terminal_general[
        where n = n and C = C and d = d and W = W
        and alpha = 1 and beta = "-lam" and delta = delta,
        OF n_pos weights last_weight carried_weight
        unit_diagonal last_diagonal initial]
      by simp
  qed

```

After all nonterminal coordinates have been eliminated, the surviving weighted identity involves only the final two linear forms and one diagonal coordinate. Choosing that coordinate to be one guarantees that the resulting rational solution is nontrivial.

```

lemma rat_weighted_elimination_nontrivial_solution:
  fixes C :: "nat ⇒ nat ⇒ rat"
  fixes d W :: "nat ⇒ rat"
  fixes lam delta :: rat

```

```

assumes n_pos:
  "0 < n"
assumes form_weights:
  " $\bigwedge q. q < n \implies W\ q = 1$ "
assumes carried_weight:
  " $W\ n = -lam$ "
assumes unit_diagonal:
  " $\bigwedge q. q < n - 1 \implies d\ q = 1$ "
assumes last_diagonal:
  " $d\ (n - 1) = delta$ "
assumes initial:
  " $\bigwedge x.$ 
     $rat\_weighted\_linear\_squares\_from$ 
       $n\ 0\ (n + 1)\ W\ C\ x$ 
     $=$ 
     $rat\_diagonal\_form\ n\ d\ x$ "
shows
  " $\exists x\ y\ z :: rat.$ 
     $(x \neq 0 \vee y \neq 0 \vee z \neq 0) \wedge$ 
     $x^2 = delta * y^2 + lam * z^2$ "
proof -
  let ?u = "rat_unit_coordinate (n - 1)"
  let ?x = "rat_linear_form n (rat_elimination_coeffs n (n - 1) C (n - 1)) ?u"
  let ?z = "rat_linear_form n (rat_elimination_coeffs n (n - 1) C n) ?u"
  have terminal: "?x^2 - lam * ?z^2 = delta"
    using rat_weighted_elimination_terminal[
      where n = n and C = C and d = d and W = W
      and lam = lam and delta = delta,
      OF n_pos form_weights carried_weight unit_diagonal last_diagonal
    ]
  initial,
  of ?u]
  by (simp add: rat_unit_coordinate_def)
  have equation: "?x^2 = delta * (1::rat)^2 + lam * ?z^2"
    using terminal by (simp add: diff_eq_eq)

  show ?thesis
    apply (rule exI[of _ ?x])
    apply (rule exI[of _ "1::rat"])
    apply (rule exI[of _ ?z])
    using equation by simp
qed

```

6 The case $v = 4w + 1$

For order $4 * w + 1$, the inverse four-square transformation reconstructs the original coordinates from a vector of new variables. The coefficient functions below express every incidence linear form, and the distinguished

sum of coordinates, in those new variables. Applying the generic elimination theorem gives the required rational solution with the plus sign.

```

context ordered_sym_bibd
begin

definition brc_inverse_y_block ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
    (nat  $\Rightarrow$  rat)  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "brc_inverse_y_block a b c d y h j =
    (case y_inv_of
      ((a,b,c,d),
       (y (4*h),
        y (4*h+1),
        y (4*h+2),
        y (4*h+3)))
    of (t0,t1,t2,t3)  $\Rightarrow$ 
      if j = 0 then t0
      else if j = 1 then t1
      else if j = 2 then t2
      else t3)"

```

```

definition brc_x_from_y_plus ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
    nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat mat" where
  "brc_x_from_y_plus a b c d w y =
    column_matrix v
      ( $\lambda$ i.
        if i < 4*w then
          brc_inverse_y_block
            a b c d y (i div 4) (i mod 4)
        else
          y i)"

```

```

lemma brc_x_from_y_plus_block:
  assumes v_form:
    "v = 4 * w + 1"
  assumes h_lt:
    "h < w"
  assumes j_lt:
    "j < 4"
  shows
    "brc_x_from_y_plus a b c d w y
      $$v (4*h+j)
    =
      brc_inverse_y_block a b c d y h j"
proof -
  have index_lt:
    "4*h+j < v"
  using v_form h_lt j_lt

```

```

    by simp

have before_last:
  "4*h+j < 4*w"
  using h_lt j_lt
  by simp

have div:
  "(4*h+j) div 4 = h"
  using j_lt
  by simp

have mod:
  "(4*h+j) mod 4 = j"
  using j_lt
  by simp

show ?thesis
  unfolding brc_x_from_y_plus_def
  using index_lt before_last div mod
  by simp
qed

lemma brc_x_from_y_plus_last:
  assumes v_form:
    "v = 4 * w + 1"
  shows
    "brc_x_from_y_plus a b c d w y
     $$$v (4*w)
     =
     y (4*w)"
  proof -
    have index_lt:
      "4*w < v"
      using v_form
      by simp

    show ?thesis
      unfolding brc_x_from_y_plus_def
      using index_lt
      by simp
  qed

lemma brc_inverse_y_block_tuple:
  "(brc_inverse_y_block a b c d y h 0,
    brc_inverse_y_block a b c d y h 1,
    brc_inverse_y_block a b c d y h 2,
    brc_inverse_y_block a b c d y h 3)
  ="

```

```

y_inv_of
  ((a,b,c,d),
   (y (4*h),
    y (4*h+1),
    y (4*h+2),
    y (4*h+3))))"
unfolding brc_inverse_y_block_def
by (cases
    "y_inv_of
     ((a,b,c,d),
      (y (4*h),
       y (4*h+1),
       y (4*h+2),
       y (4*h+3))))"
    simp

lemma brc_inverse_y_block_forward:
  assumes nz:
    "a^2 + b^2 + c^2 + d^2 ≠ 0"
  shows
    "y_of
     ((a,b,c,d),
      (brc_inverse_y_block a b c d y h 0,
       brc_inverse_y_block a b c d y h 1,
       brc_inverse_y_block a b c d y h 2,
       brc_inverse_y_block a b c d y h 3))
    =
     (y (4*h),
      y (4*h+1),
      y (4*h+2),
      y (4*h+3))"
proof -
  have inverse:
    "y_of
     ((a,b,c,d),
      y_inv_of
        ((a,b,c,d),
         (y (4*h),
          y (4*h+1),
          y (4*h+2),
          y (4*h+3))))"
    =
     (y (4*h),
      y (4*h+1),
      y (4*h+2),
      y (4*h+3))"
  using four_square_transform_inverse[
    OF nz,
    of "y (4*h)"

```

```

      "y (4*h+1)"
      "y (4*h+2)"
      "y (4*h+3)"]
    by simp

show ?thesis
  using inverse brc_inverse_y_block_tuple[
    where a = a and b = b and c = c and d = d
    and y = y and h = h]
  by simp
qed

lemma brc_x_from_y_plus_block_sqsum:
  assumes v_form:
    "v = 4 * w + 1"
  assumes h_lt:
    "h < w"
  assumes nz:
    "a^2 + b^2 + c^2 + d^2 ≠ 0"
  shows
    "y_block_sqsum a b c d
      (brc_x_from_y_plus a b c d w y) h
    =
      (y (4*h))^2 +
      (y (4*h+1))^2 +
      (y (4*h+2))^2 +
      (y (4*h+3))^2"
  proof -
    let ?x =
      "brc_x_from_y_plus a b c d w y"

    have x0:
      "?x $$$v (4*h)"
      =
      brc_inverse_y_block a b c d y h 0"
    using brc_x_from_y_plus_block[
      where w = w and h = h and j = 0
      and a = a and b = b and c = c and d = d
      and y = y,
      OF v_form h_lt]
    by simp

    have x1:
      "?x $$$v (4*h+1)"
      =
      brc_inverse_y_block a b c d y h 1"
    using brc_x_from_y_plus_block[
      where w = w and h = h and j = 1
      and a = a and b = b and c = c and d = d

```

```

      and y = y,
      OF v_form h_lt]
    by simp

have x2:
  "?x $$$v (4*h+2)
  =
  brc_inverse_y_block a b c d y h 2"
using brc_x_from_y_plus_block[
  where w = w and h = h and j = 2
  and a = a and b = b and c = c and d = d
  and y = y,
  OF v_form h_lt]
by simp

have x3:
  "?x $$$v (4*h+3)
  =
  brc_inverse_y_block a b c d y h 3"
using brc_x_from_y_plus_block[
  where w = w and h = h and j = 3
  and a = a and b = b and c = c and d = d
  and y = y,
  OF v_form h_lt]
by simp

have forward:
  "y_of
  ((a,b,c,d),
  (?x $$$v (4*h),
  ?x $$$v (4*h+1),
  ?x $$$v (4*h+2),
  ?x $$$v (4*h+3)))
  =
  (y (4*h),
  y (4*h+1),
  y (4*h+2),
  y (4*h+3))"
using brc_inverse_y_block_forward[
  where a = a and b = b and c = c and d = d
  and y = y and h = h,
  OF nz]
  x0 x1 x2 x3
by simp

show ?thesis
  unfolding y_block_sqsum_def
  using forward
  by simp

```

qed

```

lemma brc_x_from_y_plus_blocks_sqsum:
  assumes v_form:
    "v = 4 * w + 1"
  assumes nz:
    "a^2 + b^2 + c^2 + d^2 ≠ 0"
  shows
    "y_blocks_sqsum a b c d
      (brc_x_from_y_plus a b c d w y) w
    =
      (∑ i∈{0..<4*w}. (y i)^2)"
proof -
  have blocks:
    "y_blocks_sqsum a b c d
      (brc_x_from_y_plus a b c d w y) w
    =
      (∑ h∈{0..

```

```

have regroup:
  "(∑ i∈{0..<4*w}. (y i)^2)
  =
  (∑ h∈{0..

```

```

show ?thesis
  using blocks regroup
  by simp

```

qed

```

definition brc_plus_coeff ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
  nat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "brc_plus_coeff a b c d w r j =
    (if r < v then
      brc_L
        (brc_x_from_y_plus
          a b c d w
          (rat_unit_coordinate j))
      r
    else
      brc_y0
        (brc_x_from_y_plus
          a b c d w
          (rat_unit_coordinate j))
      w)"

```

```

definition brc_plus_weight ::
  "nat  $\Rightarrow$  rat" where
  "brc_plus_weight r =
    (if r < v then 1 else - of_nat  $\Lambda$ )"

```

```

definition brc_plus_diagonal ::
  "nat  $\Rightarrow$  rat" where
  "brc_plus_diagonal j =
    (if j < v - 1 then
      1
    else
      of_nat (k -  $\Lambda$ ))"

```

```

lemma brc_plus_weight_L:
  assumes r_lt:
    "r < v"
  shows
    "brc_plus_weight r = 1"
  using r_lt
  unfolding brc_plus_weight_def
  by simp

```

```

lemma brc_plus_weight_y0:
  "brc_plus_weight v = - of_nat  $\Lambda$ "
  unfolding brc_plus_weight_def
  by simp

```

```

lemma brc_plus_diagonal_unit:
  assumes j_lt:
    "j < v - 1"

```

```

shows
  "brc_plus_diagonal j = 1"
using j_lt
unfolding brc_plus_diagonal_def
by simp

lemma brc_plus_diagonal_last:
  assumes v_pos:
    "0 < v"
  shows
    "brc_plus_diagonal (v - 1)
     =
     of_nat (k -  $\Lambda$ )"
  using v_pos
  unfolding brc_plus_diagonal_def
  by simp

lemma brc_inverse_y_block_add:
  "brc_inverse_y_block a b c d
   ( $\lambda i. y\ i + z\ i$ ) h j
   =
   brc_inverse_y_block a b c d y h j
   +
   brc_inverse_y_block a b c d z h j"
  unfolding brc_inverse_y_block_def
  apply (cases "j = 0")
  apply simp
  apply (subst add_divide_distrib[symmetric])
  apply (simp add: algebra_simps)
  apply (cases "j = 1")
  apply simp
  apply (subst add_divide_distrib[symmetric])
  apply (simp add: algebra_simps)
  apply (cases "j = 2")
  apply simp
  apply (subst add_divide_distrib[symmetric])
  apply (simp add: algebra_simps)
  apply simp
  apply (subst add_divide_distrib[symmetric])
  apply (simp add: algebra_simps)
  done

lemma brc_inverse_y_block_scale:
  "brc_inverse_y_block a b c d
   ( $\lambda i. u * y\ i$ ) h j
   =
   u * brc_inverse_y_block a b c d y h j"
  unfolding brc_inverse_y_block_def
  by (auto simp:

```

```

    algebra_simps)

lemma brc_x_from_y_plus_add:
  assumes i_bound:
    "i < v"
  shows
    "brc_x_from_y_plus a b c d w
      ( $\lambda j. y\ j + z\ j$ )  $\$ \$_v$  (i)
    =
    brc_x_from_y_plus a b c d w y  $\$ \$_v$  (i)
    +
    brc_x_from_y_plus a b c d w z  $\$ \$_v$  (i)"
proof (cases "i < 4*w")
  case True

  show ?thesis
    unfolding brc_x_from_y_plus_def
    using i_bound True
    brc_inverse_y_block_add[
      where a = a and b = b and c = c and d = d
      and y = y and z = z
      and h = "i div 4" and j = "i mod 4"]
    by simp
next
  case False

  show ?thesis
    unfolding brc_x_from_y_plus_def
    using i_bound False
    by simp
qed

lemma brc_x_from_y_plus_scale:
  assumes i_bound:
    "i < v"
  shows
    "brc_x_from_y_plus a b c d w
      ( $\lambda j. u * y\ j$ )  $\$ \$_v$  (i)
    =
    u *
    brc_x_from_y_plus a b c d w y  $\$ \$_v$  (i)"
proof (cases "i < 4*w")
  case True

  show ?thesis
    unfolding brc_x_from_y_plus_def
    using i_bound True
    brc_inverse_y_block_scale[
      where a = a and b = b and c = c and d = d

```

```

        and u = u and y = y
        and h = "i div 4" and j = "i mod 4"]
    by simp
next
  case False

  show ?thesis
    unfolding brc_x_from_y_plus_def
    using i_bound False
    by simp
qed

lemma brc_x_from_y_plus_sum:
  assumes i_bound: "i < v"
  shows
    "brc_x_from_y_plus a b c d w
     ( $\sum_{j \in S. f\ j}$ )  $$$$_v\ (i)$ 
    =
    ( $\sum_{j \in S. brc\_x\_from\_y\_plus\ a\ b\ c\ d\ w\ (f\ j)\ $$$_v\ (i)}$ )"
  by (rule rat_function_sum) (rule brc_x_from_y_plus_add[OF i_bound])

end

```

The following finite-basis expansion is elementary rational linear algebra. It is stated outside the design locale and is used below only to expand the design-specific reconstruction map.

```

definition rat_basis_expansion ::
  "nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "rat_basis_expansion n y =
    ( $\lambda t.$ 
       $\sum_{j \in \{0..<n\}. y\ j * rat\_unit\_coordinate\ j\ t}$ )"

```

```

lemma rat_basis_expansion_inside:
  assumes t_bound:
    "t < n"
  shows
    "rat_basis_expansion n y t = y t"
proof -
  have t_mem:
    "t  $\in$  {0.. $<n$ }"
    using t_bound
    by simp

  have
    "( $\sum_{j \in \{0..<n\}. y\ j * (if\ t = j\ then\ 1\ else\ 0)}$ ) =
     ( $\sum_{j \in \{0..<n\}. if\ j = t\ then\ y\ j\ else\ 0}$ )"
    by (rule sum.cong) auto

```

```

also have "... = y t"
  using t_mem
  by simp
finally show ?thesis
  unfolding rat_basis_expansion_def rat_unit_coordinate_def
  .
qed

```

A linear map that depends only on the first n coordinates is determined by its values on the n unit vectors. Both odd-order cases use this same expansion; only their reconstruction maps and dimensions differ.

```

lemma rat_function_linear_expansion:
  fixes F :: "(nat  $\Rightarrow$  rat)  $\Rightarrow$  rat"
  assumes add: " $\bigwedge x z. F (\lambda j. x j + z j) = F x + F z$ "
    and scale: " $\bigwedge r x. F (\lambda j. r * x j) = r * F x$ "
    and basis: " $F y = F (\text{rat\_basis\_expansion } n y)$ "
  shows " $F y = (\sum_{j \in \{0..<n\}} y j * F (\text{rat\_unit\_coordinate } j))$ "
proof -
  have expansion:
    " $\text{rat\_basis\_expansion } n y =$ 
       $(\sum_{j \in \{0..<n\}} (\lambda t. y j * \text{rat\_unit\_coordinate } j t))$ "
    unfolding rat_basis_expansion_def
    using finite_sum_functions[
      where S = "{0..<n}" and f = " $\lambda j t. y j * \text{rat\_unit\_coordinate } j$ 
t"]
    by simp
  have "F y = F (rat_basis_expansion n y)"
    by (rule basis)
  also have "... =  $(\sum_{j \in \{0..<n\}} F (\lambda t. y j * \text{rat\_unit\_coordinate } j t))$ "
    unfolding expansion
    by (rule rat_function_sum[
      where F = F
      and S = "{0..<n}"
      and f = " $\lambda j. \lambda t. y j * \text{rat\_unit\_coordinate } j t$ ",
      OF add])
  also have "... =  $(\sum_{j \in \{0..<n\}} y j * F (\text{rat\_unit\_coordinate } j))$ "
    by (simp only: scale)
  finally show ?thesis .
qed

```

```

context ordered_sym_bibd
begin

```

```

lemma brc_x_from_y_plus_cong:
  assumes v_form:
    " $v = 4 * w + 1$ "
  assumes i_bound:
    " $i < v$ "
  assumes agree:

```

```

    "\^t. t < v ==> y t = z t"
  shows
    "brc_x_from_y_plus a b c d w y $$$v (i)
    =
    brc_x_from_y_plus a b c d w z $$$v (i)"
proof (cases "i < 4*w")
  case True

  let ?h = "i div 4"

  have h_lt:
    "?h < w"
  using True
  by (simp add: div_less_iff_less_mult)

  have b0:
    "4*?h < v"
  using h_lt v_form
  by simp

  have b1:
    "4*?h+1 < v"
  using h_lt v_form
  by simp

  have b2:
    "4*?h+2 < v"
  using h_lt v_form
  by simp

  have b3:
    "4*?h+3 < v"
  using h_lt v_form
  by simp

  have y0:
    "y (4*?h) = z (4*?h)"
  using agree[OF b0] .

  have y1:
    "y (4*?h+1) = z (4*?h+1)"
  using agree[OF b1] .

  have y2:
    "y (4*?h+2) = z (4*?h+2)"
  using agree[OF b2] .

  have y3:
    "y (4*?h+3) = z (4*?h+3)"

```

```

    using agree[OF b3] .

show ?thesis
  unfolding brc_x_from_y_plus_def
    brc_inverse_y_block_def
  using i_bound True y0 y1 y2 y3
  by simp
next
case False

have yi:
  "y i = z i"
  using agree[OF i_bound] .

show ?thesis
  unfolding brc_x_from_y_plus_def
  using i_bound False yi
  by simp
qed

lemma brc_x_from_y_plus_basis:
  assumes v_form:
    "v = 4 * w + 1"
  assumes i_bound:
    "i < v"
  shows
    "brc_x_from_y_plus a b c d w y $$$v (i)
    =
    brc_x_from_y_plus a b c d w
    (rat_basis_expansion v y) $$$v (i)"
proof -
  have agree:
    " $\bigwedge t. t < v \implies$ 
    y t = rat_basis_expansion v y t"
  proof -
    fix t
    assume t_bound:
      "t < v"

    show
      "y t = rat_basis_expansion v y t"
    using rat_basis_expansion_inside[
      where n = v and y = y and t = t,
      OF t_bound]
    by simp
  qed

show ?thesis
  using brc_x_from_y_plus_cong[

```

```

      where i = i
            and a = a and b = b and c = c and d = d
            and w = w
            and y = y
            and z = "rat_basis_expansion v y",
      OF v_form i_bound agree]
.
qed

lemma brc_x_from_y_plus_linear_expansion:
  assumes v_form:
    "v = 4 * w + 1"
  assumes i_bound:
    "i < v"
  shows
    "brc_x_from_y_plus a b c d w y $$$v (i)
    =
    (∑ j∈{0..v (i))"
  apply (rule rat_function_linear_expansion)
  apply (rule brc_x_from_y_plus_add[OF i_bound])
  apply (rule brc_x_from_y_plus_scale[OF i_bound])
  by (rule brc_x_from_y_plus_basis[OF v_form i_bound])

lemma brc_plus_coeff_L_representation:
  assumes v_form:
    "v = 4 * w + 1"
  assumes r_bound:
    "r < v"
  shows
    "brc_L
    (brc_x_from_y_plus a b c d w y) r
    =
    rat_linear_form v
    (brc_plus_coeff a b c d w r)
    y"
  proof -
    let ?X =
      "brc_x_from_y_plus a b c d w"

    have entry:
      "?X y $$$v (h)
      =
      (∑ j∈{0..v (h))"
      if h_bound:

```

```

    "h ∈ {0..v (h)))"
  unfolding brc_L_def
  using entry
  by (intro sum.cong) auto

also have
  "... =
  (∑ h∈{0..v (h)))"
  by (simp only: sum_distrib_left)

also have
  "... =
  (∑ j∈{0..v (h)))"
  by (rule sum.swap)

also have
  "... =
  (∑ j∈{0..v (h)))"
  apply (rule sum.cong)
  apply (rule refl)

```

```

    apply (simp add:
      sum_distrib_left
      mult.assoc
      mult.left_commute
      mult.commute)
  done

also have
  "... =
    ( $\sum_{j \in \{0..v\}} y_j * brc_L$ 
      ( $?X (rat\_unit\_coordinate\ j))\ r$ )"
  unfolding brc_L_def
  by simp

also have
  "... =
    ( $\sum_{j \in \{0..v\}} brc\_plus\_coeff\ a\ b\ c\ d\ w\ r\ j * y_j$ )"
  proof -
    show ?thesis
      apply (rule sum.cong)
      apply (rule refl)
    using r_bound
      unfolding brc_plus_coeff_def
      by (simp add: mult.commute)
  qed

also have
  "... =
    rat_linear_form v
      (brc_plus_coeff a b c d w r)
    y"
  unfolding rat_linear_form_def
  by simp

  finally show ?thesis .
qed

lemma brc_y0_as_full_sum:
  assumes v_form:
    " $v = 4 * w + 1$ "
  shows
    " $brc\_y0\ x\ w$ 
      =
      ( $\sum_{h \in \{0..v\}} x\ \$\$_v\ (h)$ )"
  proof -

```

```

have split:
  "(\sum h \in \{0..<v\}. x $$$_v (h))
  =
  (\sum h \in \{0..<4*w\}. x $$$_v (h))
  + x $$$_v (4*w)"
using brc_x_sum_split_last[
  OF v_form, of x]
.

show ?thesis
  unfolding brc_y0_def
    x_head_sum_def x_last_def
  using split
  by simp
qed

lemma brc_plus_coeff_y0_representation:
  assumes v_form:
    "v = 4 * w + 1"
  shows
    "brc_y0
     (brc_x_from_y_plus a b c d w y) w
    =
    rat_linear_form v
     (brc_plus_coeff a b c d w v)
     y"
proof -
  let ?X =
    "brc_x_from_y_plus a b c d w"

  have entry:
    "?X y $$$_v (h)
    =
    (\sum j \in \{0..<v\}.
     y j *
     ?X (rat_unit_coordinate j) $$$_v (h))"
  if h_mem:
    "h \in \{0..<v\}"
  for h
  using brc_x_from_y_plus_linear_expansion[
    where i = h
      and a = a and b = b and c = c and d = d
      and w = w and y = y,
    OF v_form]
    h_mem
  by simp

  have expanded:
    "brc_y0 (?X y) w

```

```

=
  (∑h∈{0..<v}.
    ∑j∈{0..<v}.
      y j *
      ?X (rat_unit_coordinate j) $$$v (h))"
  unfolding brc_y0_as_full_sum[OF v_form]
  using entry
  by (intro sum.cong) auto

also have
  "... =
    (∑j∈{0..<v}.
      ∑h∈{0..<v}.
        y j *
        ?X (rat_unit_coordinate j) $$$v (h))"
  by (rule sum.swap)

also have
  "... =
    (∑j∈{0..<v}.
      y j *
      (∑h∈{0..<v}.
        ?X (rat_unit_coordinate j) $$$v (h)))"
  proof (rule sum.cong)
    show "{0..<v} = {0..<v}"
      by simp
  next
    fix j
    assume j_mem:
      "j ∈ {0..<v}"

    show
      "(∑h∈{0..<v}.
        y j *
        ?X (rat_unit_coordinate j) $$$v (h))
      =
        y j *
        (∑h∈{0..<v}.
          ?X (rat_unit_coordinate j) $$$v (h))"
      by (metis sum_distrib_left)
  qed

also have
  "... =
    (∑j∈{0..<v}.
      y j *
      brc_y0
      (?X (rat_unit_coordinate j)) w)"
  using brc_y0_as_full_sum[OF v_form]

```

```

    by simp

also have
  "... =
    (∑ j ∈ {0..<v}.
      brc_plus_coeff a b c d w v j *
      y j)"
  unfolding brc_plus_coeff_def
  by (intro sum.cong) (auto simp: mult.commute)

also have
  "... =
    rat_linear_form v
      (brc_plus_coeff a b c d w v)
      y"
  unfolding rat_linear_form_def
  by simp

  finally show ?thesis .
qed

lemma brc_plus_weighted_initial:
  assumes v_form:
    "v = 4 * w + 1"
  assumes abcd:
    "k - Λ = a^2 + b^2 + c^2 + d^2"
  shows
    "∧ y.
      rat_weighted_linear_squares_from
        v 0 (v + 1)
        brc_plus_weight
          (brc_plus_coeff a b c d w)
          y
      =
      rat_diagonal_form v
        brc_plus_diagonal y"
proof -
  fix y

  let ?X =
    "brc_x_from_y_plus a b c d w y"

  have diff_nz:
    "k - Λ ≠ 0"
  using block_size_gt_index
  by simp

  have nz:
    "a^2 + b^2 + c^2 + d^2 ≠ 0"

```

```

using abcd diff_nz
by metis

have transformed:
  "(\sum r \in \{0..<v\}. (brc_L ?X r)^2)
  =
  of_nat \Lambda * (brc_y0 ?X w)^2
  +
  y_blocks_sqsum a b c d ?X w
  +
  of_nat (k - \Lambda) * (brc_yv ?X w)^2"
using brc_x_equation_in_linear_forms[
  where w = w and a = a and b = b and c = c and d = d
  and x = ?X,
  OF v_form abcd]
.

have blocks:
  "y_blocks_sqsum a b c d ?X w
  =
  (\sum j \in \{0..<v - 1\}. (y j)^2)"
proof -
  have recovered:
    "y_blocks_sqsum a b c d ?X w
    =
    (\sum j \in \{0..<4*w\}. (y j)^2)"
  using brc_x_from_y_plus_blocks_sqsum[
    where w = w and a = a and b = b
    and c = c and d = d and y = y,
    OF v_form nz]
  .

have range:
  "4*w = v - 1"
  using v_form
  by simp

show ?thesis
  using recovered range
  by simp
qed

have last:
  "brc_yv ?X w = y (v - 1)"
proof -
  have entry:
    "?X $$_v (4*w) = y (4*w)"
  using brc_x_from_y_plus_last[
    where w = w and a = a and b = b

```

```

        and c = c and d = d and y = y,
      OF v_form]
    .

show ?thesis
  unfolding brc_yv_def x_last_def
  using entry v_form
  by simp
qed

have L_forms:
  "( $\sum_{r \in \{0..<v\}}.$ 
    (rat_linear_form v
      (brc_plus_coeff a b c d w r) y)^2)
  =
  ( $\sum_{r \in \{0..<v\}}.$  (brc_L ?X r)^2)"
proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
  using brc_plus_coeff_L_representation[
    where w = w and a = a and b = b
    and c = c and d = d and y = y,
    OF v_form]
  by auto
qed

have y0_form:
  "rat_linear_form v
    (brc_plus_coeff a b c d w v) y
  =
  brc_y0 ?X w"
  using brc_plus_coeff_y0_representation[
    where w = w and a = a and b = b
    and c = c and d = d and y = y,
    OF v_form]
  by simp

have weighted:
  "rat_weighted_linear_squares_from
    v 0 (v + 1)
    brc_plus_weight
    (brc_plus_coeff a b c d w) y
  =
  ( $\sum_{r \in \{0..<v\}}.$  (brc_L ?X r)^2)
  -
  of_nat  $\Lambda$  * (brc_y0 ?X w)^2"
proof -
  have split:

```

```

"( $\sum_{r \in \{0..<v + 1\}}.$ 
  brc_plus_weight r *
  (rat_linear_form v
    (brc_plus_coeff a b c d w r) y)^2)
=
( $\sum_{r \in \{0..<v\}}.$ 
  brc_plus_weight r *
  (rat_linear_form v
    (brc_plus_coeff a b c d w r) y)^2)
+
brc_plus_weight v *
  (rat_linear_form v
    (brc_plus_coeff a b c d w v) y)^2"
by (simp add: sum.atLeastLessThan_Suc)

show ?thesis
  unfolding rat_weighted_linear_squares_from_def
  using split L_forms y0_form
  unfolding brc_plus_weight_def
  by simp
qed

have diagonal:
  "rat_diagonal_form v brc_plus_diagonal y
  =
  ( $\sum_{j \in \{0..<v - 1\}}.$  (y j)^2)
  +
  of_nat (k -  $\Lambda$ ) * (y (v - 1))^2"
proof -
  have v_pos:
    "0 < v"
  using v_form
  by simp

  obtain t where v_eq:
    "v = Suc t"
  using v_pos
  by (cases v) auto

  show ?thesis
    unfolding rat_diagonal_form_def
      brc_plus_diagonal_def
    using v_eq
    by (simp add: sum.atLeastLessThan_Suc)
qed

have rearranged:
  " $\sum_{r \in \{0..<v\}}.$  (brc_L ?X r)^2)
  -

```

```

    of_nat  $\Lambda$  * (brc_y0 ?X w)^2
  =
  ( $\sum_{j \in \{0..<v-1\}} (y\ j)^2$ )
  +
  of_nat (k -  $\Lambda$ ) * (y (v - 1))^2"
proof -
  have
    " ( $\sum_{r \in \{0..<v\}} (brc_L\ ?X\ r)^2$ )
    =
    of_nat  $\Lambda$  * (brc_y0 ?X w)^2
    +
    ( $\sum_{j \in \{0..<v-1\}} (y\ j)^2$ )
    +
    of_nat (k -  $\Lambda$ ) * (y (v - 1))^2"
    using transformed_blocks last
    by simp

  then show ?thesis
    by simp
qed

show
  "rat_weighted_linear_squares_from
   v 0 (v + 1)
   brc_plus_weight
   (brc_plus_coeff a b c d w) y
  =
  rat_diagonal_form v
   brc_plus_diagonal y"
  using weighted_diagonal_rearranged
  by simp
qed

lemma brc_plus_rational_solution:
  assumes v_form:
    "v = 4 * w + 1"
  assumes abcd:
    "k -  $\Lambda$  = a^2 + b^2 + c^2 + d^2"
  shows
    " $\exists x\ y\ z :: \text{rat.}$ 
     (x  $\neq$  0  $\vee$  y  $\neq$  0  $\vee$  z  $\neq$  0)  $\wedge$ 
     x^2 =
     of_nat (k -  $\Lambda$ ) * y^2 +
     of_nat  $\Lambda$  * z^2"
proof -
  have v_pos:
    "0 < v"
  using v_non_zero
  by simp

```

```

have weights:
  " $\bigwedge q. q < v \implies \text{brc\_plus\_weight } q = 1$ "
  using brc_plus_weight_L
  by blast

have carried:
  " $\text{brc\_plus\_weight } v = - \text{of\_nat } \Lambda$ "
  using brc_plus_weight_y0 .

have units:
  " $\bigwedge q. q < v - 1 \implies$ 
     $\text{brc\_plus\_diagonal } q = 1$ "
  using brc_plus_diagonal_unit
  by blast

have last:
  " $\text{brc\_plus\_diagonal } (v - 1)$ 
    =
     $\text{of\_nat } (k - \Lambda)$ "
  using brc_plus_diagonal_last[
    OF v_pos]
  .

have initial:
  " $\bigwedge u.$ 
     $\text{rat\_weighted\_linear\_squares\_from}$ 
       $v \ 0 \ (v + 1)$ 
       $\text{brc\_plus\_weight}$ 
       $(\text{brc\_plus\_coeff } a \ b \ c \ d \ w)$ 
       $u$ 
    =
     $\text{rat\_diagonal\_form } v$ 
       $\text{brc\_plus\_diagonal } u$ "
  using brc_plus_weighted_initial[
    OF v_form abcd]
  .

show ?thesis
  using rat_weighted_elimination_nontrivial_solution[
    where n = v
    and C = " $\text{brc\_plus\_coeff } a \ b \ c \ d \ w$ "
    and d =  $\text{brc\_plus\_diagonal}$ 
    and W =  $\text{brc\_plus\_weight}$ 
    and lam = " $\text{of\_nat } \Lambda$ "
    and delta = " $\text{of\_nat } (k - \Lambda)$ ",
    OF v_pos weights carried units last initial]
  .
qed

```

end

context ordered_sym_bibd
begin

7 The case $v = 4w - 1$

For order $4 * w - 1$, one auxiliary coordinate completes the coordinates to blocks of four. Its diagonal coefficient has the opposite sign, which produces the minus sign in the terminal Bruck-Ryser-Chowla equation.

```
definition brc_x_from_y_minus ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
  nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat mat" where
  "brc_x_from_y_minus a b c d w y =
    column_matrix v
    ( $\lambda$ i.
      brc_inverse_y_block
        a b c d y
        (i div 4) (i mod 4))"
```

```
definition brc_xv1_from_y_minus ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
  nat  $\Rightarrow$  (nat  $\Rightarrow$  rat)  $\Rightarrow$  rat" where
  "brc_xv1_from_y_minus a b c d w y =
    brc_inverse_y_block
      a b c d y
      ((v) div 4) ((v) mod 4)"
```

```
definition brc_minus_coeff ::
  "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
  nat  $\Rightarrow$  nat  $\Rightarrow$  rat" where
  "brc_minus_coeff a b c d w r j =
    (if r < v then
      brc_L
        (brc_x_from_y_minus
          a b c d w
          (rat_unit_coordinate j))
      r
    else if r = v then
      brc_xv1_from_y_minus
        a b c d w
        (rat_unit_coordinate j)
    else
      t_of
        (brc_x_from_y_minus
          a b c d w
          (rat_unit_coordinate j)))"
```

```

definition brc_minus_diagonal ::
  "nat  $\Rightarrow$  rat" where
  "brc_minus_diagonal j = 1"

```

```

definition brc_minus_weight ::
  "nat  $\Rightarrow$  rat" where
  "brc_minus_weight r =
    (if r < v then
      1
    else if r = v then
      of_nat (k -  $\Lambda$ )
    else
      - of_nat  $\Lambda$ )"

```

```

lemma brc_minus_weight_L:
  assumes r_lt:
    "r < v"
  shows
    "brc_minus_weight r = 1"
  using r_lt
  unfolding brc_minus_weight_def
  by simp

```

```

lemma brc_minus_weight_xv1:
  "brc_minus_weight v =
    of_nat (k -  $\Lambda$ )"
  unfolding brc_minus_weight_def
  by simp

```

```

lemma brc_minus_weight_y0:
  "brc_minus_weight (v + 1) =
    - of_nat  $\Lambda$ "
  unfolding brc_minus_weight_def
  by simp

```

```

lemma brc_minus_diagonal_one:
  "brc_minus_diagonal j = 1"
  unfolding brc_minus_diagonal_def
  by simp

```

```

lemma brc_x_from_y_minus_add:
  assumes i_bound:
    "i < v"
  shows
    "brc_x_from_y_minus a b c d w
      ( $\lambda j. y\ j + z\ j$ )  $\$ \$_v$  (i)
    =
      brc_x_from_y_minus a b c d w y  $\$ \$_v$  (i)"

```

```

+
  brc_x_from_y_minus a b c d w z $$$v (i)"
proof -
  show ?thesis
  unfolding brc_x_from_y_minus_def
  using i_bound
  brc_inverse_y_block_add[
    where a = a and b = b and c = c and d = d
    and y = y and z = z
    and h = "i div 4" and j = "i mod 4"]
  by simp
qed

```

```

lemma brc_x_from_y_minus_scale:
  assumes i_bound:
    "i < v"
  shows
    "brc_x_from_y_minus a b c d w
      (λj. u * y j) $$$v (i)
    =
    u *
    brc_x_from_y_minus a b c d w y $$$v (i)"
proof -
  show ?thesis
  unfolding brc_x_from_y_minus_def
  using i_bound
  brc_inverse_y_block_scale[
    where a = a and b = b and c = c and d = d
    and u = u and y = y
    and h = "i div 4" and j = "i mod 4"]
  by simp
qed

```

```

lemma brc_xv1_from_y_minus_add:
  "brc_xv1_from_y_minus a b c d w
    (λj. y j + z j)
  =
  brc_xv1_from_y_minus a b c d w y
  +
  brc_xv1_from_y_minus a b c d w z"
unfolding brc_xv1_from_y_minus_def
using brc_inverse_y_block_add[
  where a = a and b = b and c = c and d = d
  and y = y and z = z
  and h = "v div 4" and j = "v mod 4"]
by simp

```

```

lemma brc_xv1_from_y_minus_scale:
  "brc_xv1_from_y_minus a b c d w

```

```

      (λj. u * y j)
    =
    u * brc_xv1_from_y_minus a b c d w y"
  unfolding brc_xv1_from_y_minus_def
  using brc_inverse_y_block_scale[
    where a = a and b = b and c = c and d = d
      and u = u and y = y
      and h = "v div 4" and j = "v mod 4"]
  by simp

definition brc_extended_x_from_y_minus ::
  "nat ⇒ nat ⇒ nat ⇒ nat ⇒ nat ⇒
  (nat ⇒ rat) ⇒ nat ⇒ rat" where
  "brc_extended_x_from_y_minus a b c d w y i =
    (if i < v then
      brc_x_from_y_minus a b c d w y $$$v (i)
    else
      brc_xv1_from_y_minus a b c d w y)"

lemma brc_extended_x_from_y_minus_add:
  assumes i_bound:
    "i < v + 1"
  shows
    "brc_extended_x_from_y_minus a b c d w
      (λj. y j + z j) i
    =
      brc_extended_x_from_y_minus a b c d w y i
    +
      brc_extended_x_from_y_minus a b c d w z i"
  proof (cases "i < v")
    case True
      show ?thesis
        unfolding brc_extended_x_from_y_minus_def
        using True
        brc_x_from_y_minus_add[
          where i = i and a = a and b = b
            and c = c and d = d and w = w
            and y = y and z = z,
          OF True]
        by simp
    next
      case False
        have i_eq:
          "i = v"
          using i_bound False
          by simp

```

```

show ?thesis
  unfolding brc_extended_x_from_y_minus_def
  using False i_eq
  brc_xv1_from_y_minus_add[
    where a = a and b = b and c = c and d = d
    and w = w and y = y and z = z]
  by simp
qed

lemma brc_extended_x_from_y_minus_scale:
  assumes i_bound:
    "i < v + 1"
  shows
    "brc_extended_x_from_y_minus a b c d w
      (λj. u * y j) i
      =
      u *
      brc_extended_x_from_y_minus a b c d w y i"
proof (cases "i < v")
  case True

  show ?thesis
    unfolding brc_extended_x_from_y_minus_def
    using True
    brc_x_from_y_minus_scale[
      where i = i and a = a and b = b
      and c = c and d = d and w = w
      and u = u and y = y,
      OF True]
    by simp
  next
  case False

  have i_eq:
    "i = v"
  using i_bound False
  by simp

  show ?thesis
    unfolding brc_extended_x_from_y_minus_def
    using False i_eq
    brc_xv1_from_y_minus_scale[
      where a = a and b = b and c = c and d = d
      and w = w and u = u and y = y]
    by simp
qed

lemma brc_extended_x_from_y_minus_sum:
  assumes i_bound: "i < v + 1"

```

```

shows
  "brc_extended_x_from_y_minus a b c d w
    ( $\sum_{j \in S}. f\ j$ ) i
  =
    ( $\sum_{j \in S}.
      \text{brc\_extended\_x\_from\_y\_minus}
        a\ b\ c\ d\ w\ (f\ j)\ i$ )"
by (rule rat_function_sum) (rule brc_extended_x_from_y_minus_add[OF i_bound])

lemma brc_inverse_y_block_cong:
  assumes e0: "y (4*h) = z (4*h)"
  assumes e1: "y (4*h+1) = z (4*h+1)"
  assumes e2: "y (4*h+2) = z (4*h+2)"
  assumes e3: "y (4*h+3) = z (4*h+3)"
  shows
    "brc_inverse_y_block a b c d y h j
    =
      brc_inverse_y_block a b c d z h j"
  using e0 e1 e2 e3
  unfolding brc_inverse_y_block_def
  by simp

lemma brc_extended_x_from_y_minus_cong:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  assumes i_bound:
    "i < v + 1"
  assumes agree:
    " $\bigwedge t. t < v + 1 \implies y\ t = z\ t$ "
  shows
    "brc_extended_x_from_y_minus a b c d w y i
    =
      brc_extended_x_from_y_minus a b c d w z i"
proof (cases "i < v")
  case True

    let ?h = "i div 4"

    have i_lt_4w:
      "i < 4*w"
      using True v_form w_pos
      by simp

    have h_lt:
      "?h < w"
      using i_lt_4w
      by (simp add: div_less_iff_less_mult)

```

```

have b0:
  "4*?h < v + 1"
  using h_lt v_form w_pos
  by simp

have b1:
  "4*?h+1 < v + 1"
  using h_lt v_form w_pos
  by simp

have b2:
  "4*?h+2 < v + 1"
  using h_lt v_form w_pos
  by simp

have b3:
  "4*?h+3 < v + 1"
  using h_lt v_form w_pos
  by simp

have block:
  "brc_inverse_y_block a b c d y ?h (i mod 4)
  =
  brc_inverse_y_block a b c d z ?h (i mod 4)"
  using brc_inverse_y_block_cong[
    where a = a and b = b and c = c and d = d
    and y = y and z = z and h = ?h and j = "i mod 4",
    OF agree[OF b0] agree[OF b1]
    agree[OF b2] agree[OF b3]]
  .

show ?thesis
  unfolding brc_extended_x_from_y_minus_def
    brc_x_from_y_minus_def
  using True block
  by simp
next
case False

have i_eq:
  "i = v"
  using i_bound False
  by simp

have v_div:
  "v div 4 = w - 1"
  using v_form w_pos
  by simp

```

```

obtain s where w_eq:
  "w = Suc s"
  using w_pos
  by (cases w) auto

have v_eq:
  "v = 4*s + 3"
  using v_form w_eq
  by simp

have v_mod:
  "v mod 4 = 3"
  using v_eq
  by simp

have b0:
  "4*(w-1) < v + 1"
  using v_form w_pos
  by simp

have b1:
  "4*(w-1)+1 < v + 1"
  using v_form w_pos
  by simp

have b2:
  "4*(w-1)+2 < v + 1"
  using v_form w_pos
  by simp

have b3:
  "4*(w-1)+3 < v + 1"
  using v_form w_pos
  by simp

have block:
  "brc_xv1_from_y_minus a b c d w y
  =
  brc_xv1_from_y_minus a b c d w z"
  unfolding brc_xv1_from_y_minus_def
  using v_div v_mod
  brc_inverse_y_block_cong[
    where a = a and b = b and c = c and d = d
    and y = y and z = z and h = "w-1" and j = 3,
    OF agree[OF b0] agree[OF b1]
    agree[OF b2] agree[OF b3]]
  by simp

```

```

show ?thesis
  unfolding brc_extended_x_from_y_minus_def
  using False i_eq block
  by simp
qed

lemma brc_extended_x_from_y_minus_basis:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  assumes i_bound:
    "i < v + 1"
  shows
    "brc_extended_x_from_y_minus a b c d w y i
    =
    brc_extended_x_from_y_minus a b c d w
    (rat_basis_expansion (v + 1) y) i"
proof -
  have agree:
    " $\bigwedge t. t < v + 1 \implies$ 
    y t =
    rat_basis_expansion (v + 1) y t"
  proof -
    fix t
    assume t_bound:
      "t < v + 1"

    show
      "y t =
      rat_basis_expansion (v + 1) y t"
    using rat_basis_expansion_inside[
      where n = "v + 1"
      and y = y and t = t,
      OF t_bound]
    by simp
  qed

show ?thesis
  using brc_extended_x_from_y_minus_cong[
    where i = i
    and a = a and b = b and c = c and d = d
    and w = w
    and y = y
    and z = "rat_basis_expansion (v + 1) y",
    OF v_form w_pos i_bound agree]
  .
qed

```

```

lemma brc_extended_x_from_y_minus_linear_expansion:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  assumes i_bound:
    "i < v + 1"
  shows
    "brc_extended_x_from_y_minus a b c d w y i
    =
    ( $\sum_{j \in \{0..<v+1\}} y^j * brc\_extended\_x\_from\_y\_minus\ a\ b\ c\ d\ w\ (rat\_unit\_coordinate\ j)\ i$ )"
  apply (rule rat_function_linear_expansion)
  apply (rule brc_extended_x_from_y_minus_add[OF i_bound])
  apply (rule brc_extended_x_from_y_minus_scale[OF i_bound])
  by (rule brc_extended_x_from_y_minus_basis[OF v_form w_pos i_bound])

lemma brc_minus_coeff_xv1_representation:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  shows
    "brc_xv1_from_y_minus a b c d w y
    =
    rat_linear_form (v + 1)
    (brc_minus_coeff a b c d w v)
    y"
  proof -
    have expansion:
      "brc_extended_x_from_y_minus a b c d w y v
      =
      ( $\sum_{j \in \{0..<v+1\}} y^j * brc\_extended\_x\_from\_y\_minus\ a\ b\ c\ d\ w\ (rat\_unit\_coordinate\ j)\ v$ )"
    using brc_extended_x_from_y_minus_linear_expansion[
      where i = v
      and a = a and b = b and c = c and d = d
      and w = w and y = y,
      OF v_form w_pos]
    by simp

  have left:
    "brc_extended_x_from_y_minus a b c d w y v
    =
    brc_xv1_from_y_minus a b c d w y"

```

```

    unfolding brc_extended_x_from_y_minus_def
    by simp

have right:
  "(\sum j \in \{0..<v + 1\}.
    y j *
    brc_extended_x_from_y_minus a b c d w
    (rat_unit_coordinate j) v)
  =
  (\sum j \in \{0..<v + 1\}.
    brc_minus_coeff a b c d w v j *
    y j)"
proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
  unfolding brc_extended_x_from_y_minus_def
    brc_minus_coeff_def
  by (simp add: mult.commute)
qed

show ?thesis
  unfolding rat_linear_form_def
  using expansion left right
  by simp
qed

lemma brc_minus_coeff_y0_representation:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  shows
    "t_of
      (brc_x_from_y_minus a b c d w y)
    =
    rat_linear_form (v + 1)
      (brc_minus_coeff a b c d w (v + 1))
      y"
proof -
  let ?E =
    "brc_extended_x_from_y_minus a b c d w"

  have original:
    "brc_x_from_y_minus a b c d w y $$$_v (h)
    =
    ?E y h"
  if h_bound:
    "h < v"

```

```

for h
using h_bound
unfolding brc_extended_x_from_y_minus_def
by simp

have entry:
  "?E y h
  =
  (∑ j∈{0..

```

```

    show "{0..<v + 1} = {0..<v + 1}"
      by simp
next
  fix j
  assume j_mem:
    "j ∈ {0..<v + 1}"

  show
    "(∑ h∈{0..<v}.
      y j * ?E (rat_unit_coordinate j) h)
    =
      y j *
      (∑ h∈{0..<v}.
        ?E (rat_unit_coordinate j) h)"
    by (metis sum_distrib_left)
qed

also have
  "... =
    (∑ j∈{0..<v + 1}.
      y j *
      t_of
        (brc_x_from_y_minus a b c d w
          (rat_unit_coordinate j)))"
proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
  unfolding t_of_def
    brc_extended_x_from_y_minus_def
  by simp
qed

also have
  "... =
    (∑ j∈{0..<v + 1}.
      brc_minus_coeff a b c d w (v + 1) j *
      y j)"
  unfolding brc_minus_coeff_def
  by (intro sum.cong) (auto simp: mult.commute)

also have
  "... =
    rat_linear_form (v + 1)
      (brc_minus_coeff a b c d w (v + 1))
    y"
  unfolding rat_linear_form_def
  by simp

```

```

    finally show ?thesis .
qed

lemma brc_minus_coeff_L_representation:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  assumes r_bound:
    "r < v"
  shows
    "brc_L
      (brc_x_from_y_minus a b c d w y) r
    =
      rat_linear_form (v + 1)
        (brc_minus_coeff a b c d w r)
      y"
proof -
  let ?E =
    "brc_extended_x_from_y_minus a b c d w"

  have entry:
    "brc_x_from_y_minus a b c d w y $$$v (h)
    =
      (∑j ∈ {0..<v + 1}.
        y j *
        ?E (rat_unit_coordinate j) h)"
  if h_mem:
    "h ∈ {0..<v}"
  for h
proof -
  have h_bound:
    "h < v"
  using h_mem
  by simp

  have original:
    "brc_x_from_y_minus a b c d w y $$$v (h)
    =
      ?E y h"
  using h_bound
  unfolding brc_extended_x_from_y_minus_def
  by simp

  have expansion:
    "?E y h
    =
      (∑j ∈ {0..<v + 1}.
        y j *

```

```

      ?E (rat_unit_coordinate j) h)"
using brc_extended_x_from_y_minus_linear_expansion[
  where i = h
    and a = a and b = b and c = c and d = d
    and w = w and y = y,
  OF v_form w_pos]
  h_bound
by simp

show ?thesis
  using original expansion
  by simp
qed

```

```

have expanded:
  "brc_L
    (brc_x_from_y_minus a b c d w y) r
  =
  (∑ h∈{0..<v}.
    of_int (N $$ (h,r)) *
    (∑ j∈{0..<v + 1}.
      y j *
      ?E (rat_unit_coordinate j) h))"
unfolding brc_L_def
using entry
by (intro sum.cong) auto

```

```

also have
  "... =
  (∑ h∈{0..<v}.
    ∑ j∈{0..<v + 1}.
      of_int (N $$ (h,r)) *
      (y j *
      ?E (rat_unit_coordinate j) h))"

```

```

proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
    apply (simp only: sum_distrib_left)
  done
qed

```

```

also have
  "... =
  (∑ j∈{0..<v + 1}.
    ∑ h∈{0..<v}.
      of_int (N $$ (h,r)) *
      (y j *
      ?E (rat_unit_coordinate j) h))"

```

```

    by (rule sum.swap)

also have
  "... =
    (∑ j∈{0..<v + 1}.
      y j *
      (∑ h∈{0..<v}.
        of_int (N $$ (h,r)) *
        ?E (rat_unit_coordinate j) h))"
proof (rule sum.cong)
  show "{0..<v + 1} = {0..<v + 1}"
    by simp
next
  fix j
  assume j_mem:
    "j ∈ {0..<v + 1}"

  show
    "(∑ h∈{0..<v}.
      of_int (N $$ (h,r)) *
      (y j * ?E (rat_unit_coordinate j) h))
    =
    y j *
    (∑ h∈{0..<v}.
      of_int (N $$ (h,r)) *
      ?E (rat_unit_coordinate j) h)"
  apply (simp only: sum_distrib_left)
  apply (rule sum.cong)
  apply (rule refl)
  apply (simp add:
    mult.assoc mult.left_commute mult.commute)
  done
qed

also have
  "... =
    (∑ j∈{0..<v + 1}.
      y j *
      brc_L
      (brc_x_from_y_minus a b c d w
        (rat_unit_coordinate j)) r)"
proof (rule sum.cong)
  show "{0..<v + 1} = {0..<v + 1}"
    by simp
next
  fix j
  assume j_mem:
    "j ∈ {0..<v + 1}"

```

```

have inner:
  "(\sum h \in \{0..<v\}.
    of_int (N $$ (h,r)) *
    ?E (rat_unit_coordinate j) h)
  =
  brc_L
    (brc_x_from_y_minus a b c d w
      (rat_unit_coordinate j)) r"
proof -
  show ?thesis
    unfolding brc_L_def
  proof (rule sum.cong)
    show "{0..<v} = {0..<v}"
      by simp
  next
    fix h
    assume h_mem:
      "h \in \{0..<v\}"

    have h_bound:
      "h < v"
      using h_mem
      by simp

    have coordinate:
      "?E (rat_unit_coordinate j) h
      =
      brc_x_from_y_minus a b c d w
        (rat_unit_coordinate j) $$$ (h)"
      using h_bound
      unfolding brc_extended_x_from_y_minus_def
      by simp

    show
      "of_int (N $$ (h,r)) *
      ?E (rat_unit_coordinate j) h
      =
      of_int (N $$ (h,r)) *
      brc_x_from_y_minus a b c d w
        (rat_unit_coordinate j) $$$ (h)"
      using coordinate
      by simp
  qed
qed

show
  "y j *
  (\sum h \in \{0..<v\}.
    of_int (N $$ (h,r)) *

```

```

      ?E (rat_unit_coordinate j) h)
    =
    y j *
    brc_L
      (brc_x_from_y_minus a b c d w
       (rat_unit_coordinate j)) r"
  using inner
  by simp
qed

also have
  "... =
  (∑ j ∈ {0..<v + 1}.
    brc_minus_coeff a b c d w r j *
    y j)"
  using r_bound
  unfolding brc_minus_coeff_def
  by (intro sum.cong) (auto simp: mult.commute)

also have
  "... =
  rat_linear_form (v + 1)
    (brc_minus_coeff a b c d w r)
  y"
  unfolding rat_linear_form_def
  by simp

  finally show ?thesis .
qed

definition brc_extended_matrix_from_y_minus ::
  "nat ⇒ nat ⇒ nat ⇒ nat ⇒ nat ⇒
  (nat ⇒ rat) ⇒ rat mat" where
  "brc_extended_matrix_from_y_minus a b c d w y =
  column_matrix (v + 1)
    (λ i.
      brc_inverse_y_block
        a b c d y
        (i div 4) (i mod 4))"

lemma brc_extended_matrix_minus_block:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  assumes h_lt:
    "h < w"
  assumes j_lt:
    "j < 4"

```

```

shows
  "brc_extended_matrix_from_y_minus
    a b c d w y  $\$v$  (4*h+j)
  =
    brc_inverse_y_block a b c d y h j"
proof -
  have index_lt:
    "4*h+j < v + 1"
  using v_form w_pos h_lt j_lt
  by simp

  have div:
    "(4*h+j) div 4 = h"
  using j_lt
  by simp

  have mod:
    "(4*h+j) mod 4 = j"
  using j_lt
  by simp

  show ?thesis
    unfolding brc_extended_matrix_from_y_minus_def
    using index_lt div mod
    by simp
qed

lemma brc_extended_matrix_minus_block_sqsum:
  assumes v_form:
    " $v = 4 * w - 1$ "
  assumes w_pos:
    " $0 < w$ "
  assumes h_lt:
    " $h < w$ "
  assumes nz:
    " $a^2 + b^2 + c^2 + d^2 \neq 0$ "
  shows
    "y_block_sqsum a b c d
      (brc_extended_matrix_from_y_minus
        a b c d w y) h
    =
      (y (4*h))^2 +
      (y (4*h+1))^2 +
      (y (4*h+2))^2 +
      (y (4*h+3))^2"
proof -
  let ?X =
    "brc_extended_matrix_from_y_minus
      a b c d w y"

```

```

have x0:
  "?X $$$v (4*h)
  =
  brc_inverse_y_block a b c d y h 0"
using brc_extended_matrix_minus_block[
  where w = w and h = h and j = 0
  and a = a and b = b and c = c and d = d
  and y = y,
  OF v_form w_pos h_lt]
by simp

have x1:
  "?X $$$v (4*h+1)
  =
  brc_inverse_y_block a b c d y h 1"
using brc_extended_matrix_minus_block[
  where w = w and h = h and j = 1
  and a = a and b = b and c = c and d = d
  and y = y,
  OF v_form w_pos h_lt]
by simp

have x2:
  "?X $$$v (4*h+2)
  =
  brc_inverse_y_block a b c d y h 2"
using brc_extended_matrix_minus_block[
  where w = w and h = h and j = 2
  and a = a and b = b and c = c and d = d
  and y = y,
  OF v_form w_pos h_lt]
by simp

have x3:
  "?X $$$v (4*h+3)
  =
  brc_inverse_y_block a b c d y h 3"
using brc_extended_matrix_minus_block[
  where w = w and h = h and j = 3
  and a = a and b = b and c = c and d = d
  and y = y,
  OF v_form w_pos h_lt]
by simp

have forward:
  "y_of
  ((a,b,c,d),
  (?X $$$v (4*h),

```

```

      ?X $$v (4*h+1),
      ?X $$v (4*h+2),
      ?X $$v (4*h+3)))
    =
    (y (4*h), y (4*h+1),
     y (4*h+2), y (4*h+3))"
  using brc_inverse_y_block_forward[
    where a = a and b = b and c = c and d = d
    and y = y and h = h,
    OF nZ]
    x0 x1 x2 x3
  by simp

show ?thesis
  unfolding y_block_sqsum_def
  using forward
  by simp
qed

lemma brc_extended_matrix_minus_eq_extend:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  assumes i_bound:
    "i < v + 1"
  shows
    "brc_extended_matrix_from_y_minus
     a b c d w y $$v (i)
    =
    brc_extend_x
      (brc_x_from_y_minus a b c d w y)
      (brc_xv1_from_y_minus a b c d w y)
      $$v (i)"
proof (cases "i < v")
  case True

  have left:
    "brc_extended_matrix_from_y_minus
     a b c d w y $$v (i)
    =
    brc_inverse_y_block
      a b c d y (i div 4) (i mod 4)"
  unfolding brc_extended_matrix_from_y_minus_def
  using i_bound
  by simp

  have right:
    "brc_x_from_y_minus a b c d w y $$v (i)

```

```

    =
    brc_inverse_y_block
      a b c d y (i div 4) (i mod 4)"
  unfolding brc_x_from_y_minus_def
  using True
  by simp

show ?thesis
  unfolding brc_extend_x_def
  using True left right
  by simp
next
case False

have i_eq:
  "i = v"
  using i_bound False
  by simp

have left:
  "brc_extended_matrix_from_y_minus
    a b c d w y $$$v (v)
  =
  brc_xv1_from_y_minus a b c d w y"
  unfolding brc_extended_matrix_from_y_minus_def
    brc_xv1_from_y_minus_def
  using v_form w_pos
  by simp

show ?thesis
  unfolding brc_extend_x_def
  using i_eq left
  by simp
qed

lemma brc_extended_matrix_minus_blocks_sqsum:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  assumes nz:
    "a2 + b2 + c2 + d2 ≠ 0"
  shows
    "y_blocks_sqsum a b c d
      (brc_extended_matrix_from_y_minus
        a b c d w y) w
    =
    (∑ i ∈ {0..<v + 1}. (y i)2)"
proof -

```

```

have blocks:
  "y_blocks_sqsum a b c d
    (brc_extended_matrix_from_y_minus
      a b c d w y) w
  =
  (∑ h∈{0..<w}.
    ((y (4*h))^2 +
      (y (4*h+1))^2 +
      (y (4*h+2))^2 +
      (y (4*h+3))^2))"
proof -
  show ?thesis
    unfolding y_blocks_sqsum_def
    apply (rule sum.cong)
    apply (rule refl)
  using brc_extended_matrix_minus_block_sqsum[
    where w = w and a = a and b = b
      and c = c and d = d and y = y,
    OF v_form w_pos _ nz]
  by auto
qed

have regroup:
  "(∑ i∈{0..<4*w}. (y i)^2)
  =
  (∑ h∈{0..<w}.
    ((y (4*h))^2 +
      (y (4*h+1))^2 +
      (y (4*h+2))^2 +
      (y (4*h+3))^2))"
  using sum_four_blocks[
    where w = w and f = "λi. (y i)^2"]
  .

have range:
  "v + 1 = 4*w"
  using v_form w_pos
  by simp

show ?thesis
  using blocks regroup range
  by simp
qed

lemma brc_minus_weighted_initial:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"

```

```

assumes abcd:
  "k -  $\Lambda$  = a2 + b2 + c2 + d2"
shows
  " $\wedge$ y.
    rat_weighted_linear_squares_from
      (v + 1) 0 (v + 2)
      brc_minus_weight
      (brc_minus_coeff a b c d w)
      y
    =
    rat_diagonal_form
      (v + 1) brc_minus_diagonal y"
proof -
  fix y

  let ?x =
    "brc_x_from_y_minus a b c d w y"

  let ?xv1 =
    "brc_xv1_from_y_minus a b c d w y"

  have diff_nz:
    "k -  $\Lambda$   $\neq$  0"
  using block_size_gt_index
  by simp

  have nz:
    "a2 + b2 + c2 + d2  $\neq$  0"
  using abcd diff_nz
  by metis

  have transformed:
    "(\sum r \in \{0..<v\}. (brc_L ?x r)^2)
      + of_nat (k -  $\Lambda$ ) * ?xv1^2
    =
    of_nat  $\Lambda$  * (t_of ?x)^2
    +
    (\sum h \in \{0..<w\}.
      y_block_sqsum a b c d
      (brc_extend_x ?x ?xv1) h)"
  using brc_x_equation_minus_transformed[
    OF v_form w_pos abcd,
    of ?x ?xv1]
  unfolding brc_L_def t_of_def
  by simp

  have block_entries:
    "brc_extend_x ?x ?xv1 $$$v (i)
    =

```

```

    brc_extended_matrix_from_y_minus
      a b c d w y $$$v (i)"
  if i_bound:
    "i < v + 1"
  for i
  using brc_extended_matrix_minus_eq_extend[
    where i = i
      and w = w and a = a and b = b
      and c = c and d = d and y = y,
    OF v_form w_pos i_bound]
  by simp

have blocks:
  "(\sum h \in \{0..<w\}.
    y_block_sqsum a b c d
      (brc_extend_x ?x ?xv1) h)
  =
  (\sum i \in \{0..<v + 1\}. (y i)^2)"
proof -
  have block_eq:
    "y_block_sqsum a b c d
      (brc_extend_x ?x ?xv1) h
    =
    y_block_sqsum a b c d
      (brc_extended_matrix_from_y_minus
        a b c d w y) h"
  if h_mem:
    "h \in \{0..<w\}"
  for h
proof -
  have h_lt:
    "h < w"
  using h_mem
  by simp

  have e0:
    "brc_extend_x ?x ?xv1 $$$v (4*h)
    =
    brc_extended_matrix_from_y_minus
      a b c d w y $$$v (4*h)"
  using block_entries h_lt v_form w_pos
  by simp

  have e1:
    "brc_extend_x ?x ?xv1 $$$v (4*h+1)
    =
    brc_extended_matrix_from_y_minus
      a b c d w y $$$v (4*h+1)"
  using block_entries h_lt v_form w_pos

```

```

by simp

have e2:
  "brc_extend_x ?x ?xv1 $$$v (4*h+2)
  =
  brc_extended_matrix_from_y_minus
    a b c d w y $$$v (4*h+2)"
  using block_entries h_lt v_form w_pos
  by simp

have e3:
  "brc_extend_x ?x ?xv1 $$$v (4*h+3)
  =
  brc_extended_matrix_from_y_minus
    a b c d w y $$$v (4*h+3)"
  using block_entries h_lt v_form w_pos
  by simp

show ?thesis
  unfolding y_block_sqsum_def
  using e0 e1 e2 e3
  by simp
qed

have sum_eq:
  "(\sum h \in \{0..<w\}.
    y_block_sqsum a b c d
      (brc_extend_x ?x ?xv1) h)
  =
  y_blocks_sqsum a b c d
    (brc_extended_matrix_from_y_minus
      a b c d w y) w"
  unfolding y_blocks_sqsum_def
  using block_eq
  by (intro sum.cong) auto

show ?thesis
  using sum_eq
  brc_extended_matrix_minus_blocks_sqsum[
    where w = w and a = a and b = b
    and c = c and d = d and y = y,
    OF v_form w_pos nz]
  by simp
qed

have L_forms:
  "(\sum r \in \{0..<v\}.
    (rat_linear_form (v + 1)
      (brc_minus_coeff a b c d w r) y)^2)

```

```

=
( $\sum_{r \in \{0..<v\}} (brc\_L \ ?x \ r)^2$ )"
proof -
  show ?thesis
    apply (rule sum.cong)
    apply (rule refl)
  using brc_minus_coeff_L_representation[
    where w = w and a = a and b = b
    and c = c and d = d and y = y,
    OF v_form w_pos]
  by auto
qed

have xv1_form:
  "rat_linear_form (v + 1)
    (brc_minus_coeff a b c d w v) y
  =
  ?xv1"
  using brc_minus_coeff_xv1_representation[
    where w = w and a = a and b = b
    and c = c and d = d and y = y,
    OF v_form w_pos]
  by simp

have y0_form:
  "rat_linear_form (v + 1)
    (brc_minus_coeff a b c d w (v + 1)) y
  =
  t_of ?x"
  using brc_minus_coeff_y0_representation[
    where w = w and a = a and b = b
    and c = c and d = d and y = y,
    OF v_form w_pos]
  by simp

have weighted:
  "rat_weighted_linear_squares_from
    (v + 1) 0 (v + 2)
    brc_minus_weight
    (brc_minus_coeff a b c d w) y
  =
  ( $\sum_{r \in \{0..<v\}} (brc\_L \ ?x \ r)^2$ )
  + of_nat (k -  $\Lambda$ ) * ?xv1^2
  - of_nat  $\Lambda$  * (t_of ?x)^2"
proof -
  show ?thesis
    unfolding rat_weighted_linear_squares_from_def
    brc_minus_weight_def
    using L_forms xv1_form y0_form

```

```

    by (simp add: sum.atLeastLessThan_Suc)
qed

have diagonal:
  "rat_diagonal_form
    (v + 1) brc_minus_diagonal y
  =
    (∑ i∈{0..

```

```

by simp

have elimination_weights:
  " $\wedge q. q < (v + 1) - 1 \implies$ 
     $brc\_minus\_weight\ q = 1$ "
proof -
  fix q
  assume q_lt:
    " $q < (v + 1) - 1$ "

  have "q < v"
  using q_lt
  by simp

  then show
    " $brc\_minus\_weight\ q = 1$ "
  using brc_minus_weight_L
  by blast
qed

have first_terminal:
  " $brc\_minus\_weight\ ((v + 1) - 1)$ 
    =
     $of\_nat\ (k - \Lambda)$ "
  using brc_minus_weight_xv1
  by simp

have second_terminal:
  " $brc\_minus\_weight\ (v + 1)$ 
    =
     $- of\_nat\ \Lambda$ "
  using brc_minus_weight_y0 .

have diagonal_units:
  " $\wedge q. q < (v + 1) - 1 \implies$ 
     $brc\_minus\_diagonal\ q = 1$ "
  using brc_minus_diagonal_one
  by blast

have diagonal_last:
  " $brc\_minus\_diagonal\ ((v + 1) - 1) = 1$ "
  using brc_minus_diagonal_one .

have initial:
  " $\wedge u.$ 
     $rat\_weighted\_linear\_squares\_from$ 
       $(v + 1)\ 0\ ((v + 1) + 1)$ 
       $brc\_minus\_weight$ 
       $(brc\_minus\_coeff\ a\ b\ c\ d\ w)$ "

```

```

      u
    =
      rat_diagonal_form
      (v + 1) brc_minus_diagonal u"
using brc_minus_weighted_initial[
  OF v_form w_pos abcd]
by simp

show ?thesis
using rat_weighted_elimination_terminal_solution_general[
  where n = "v + 1"
  and C = "brc_minus_coeff a b c d w"
  and d = brc_minus_diagonal
  and W = brc_minus_weight
  and alpha = "of_nat (k - Λ)"
  and beta = "- of_nat Λ",
  OF n_pos elimination_weights
  first_terminal second_terminal
  diagonal_units diagonal_last initial]
by simp
qed

lemma brc_plus_rational_solution_exists:
  assumes v_form:
    "v = 4 * w + 1"
  shows
    "∃ x y z :: rat.
      (x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0) ∧
      x^2 =
        of_nat (k - Λ) * y^2 +
        of_nat Λ * z^2"
  proof -
    obtain a b c d :: nat where abcd:
      "k - Λ = a^2 + b^2 + c^2 + d^2"
    using sum_of_four_squares[
      of "k - Λ"]
    by blast

    show ?thesis
      using brc_plus_rational_solution[
        OF v_form abcd]
      .
  qed

lemma brc_minus_rational_solution_exists:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"

```

```

shows
  "∃ x y z :: rat.
    (x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0) ∧
    x^2 =
      of_nat (k - Λ) * y^2
      - of_nat Λ * z^2"
proof -
  obtain a b c d :: nat where abcd:
    "k - Λ = a^2 + b^2 + c^2 + d^2"
  using sum_of_four_squares[
    of "k - Λ"]
  by blast

  show ?thesis
    using brc_minus_rational_solution[
      OF v_form w_pos abcd]
    .
qed

lemma brc_v_eq_4w_minus_1:
  assumes mod3:
    "v mod 4 = 3"
  shows
    "∃ w. 0 < w ∧ v = 4 * w - 1"
proof -
  let ?q = "v div 4"
  let ?w = "Suc ?q"

  have division:
    "v = 4 * ?q + 3"
  proof -
    have
      "v div 4 * 4 + v mod 4 = v"
      using div_mult_mod_eq[
        of v 4]
      by simp

    then show ?thesis
      using mod3
      by (simp add: mult.commute)
  qed

  have w_pos:
    "0 < ?w"
  by simp

  have form:
    "v = 4 * ?w - 1"
  using division

```

```

    by simp

show ?thesis
  using w_pos form
  by blast
qed

end

```

8 Clearing denominators and assembling the theorem

The elimination argument naturally yields rational witnesses. A common denominator converts them to integer witnesses; homogeneity of the quadratic equation preserves both the equality and nontriviality. The final theorem then combines the two odd residue classes modulo four.

The next two lemmas are generic facts about rational quadratic equations. They clear denominators without using any incidence-matrix or design assumption, so they are kept outside the design locale.

```

lemma rat_as_int_quotient:
  fixes r :: rat
  obtains n d :: int where
    "d ≠ 0"
    "r = of_int n / of_int d"
proof -
  obtain n d :: int where q: "quotient_of r = (n,d)"
    by (cases "quotient_of r") auto
  have "d ≠ 0"
    using Rat.quotient_of_denom_pos[OF q] by simp
  then show thesis
    using that Rat.quotient_of_div[OF q] by blast
qed

```

```

lemma rat_quadratic_solution_to_int:
  fixes A B :: int
  fixes x y z :: rat
  assumes nonzero:
    "x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0"
  assumes equation:
    "x^2 =
      of_int A * y^2 +
      of_int B * z^2"
  shows
    "∃ X Y Z :: int.
      (X ≠ 0 ∨ Y ≠ 0 ∨ Z ≠ 0) ∧
      X^2 = A * Y^2 + B * Z^2"
proof -

```

```

obtain nx dx :: int where dx: "dx ≠ 0"
  and x_rep: "x = of_int nx / of_int dx"
  using rat_as_int_quotient[of x] by blast
obtain ny dy :: int where dy: "dy ≠ 0"
  and y_rep: "y = of_int ny / of_int dy"
  using rat_as_int_quotient[of y] by blast
obtain nz dz :: int where dz: "dz ≠ 0"
  and z_rep: "z = of_int nz / of_int dz"
  using rat_as_int_quotient[of z] by blast

let ?D = "dx * dy * dz"
let ?X = "nx * dy * dz"
let ?Y = "ny * dx * dz"
let ?Z = "nz * dx * dy"
have scale_x: "(of_int ?D :: rat) * x = of_int ?X"
  unfolding x_rep using dx by simp
have scale_y: "(of_int ?D :: rat) * y = of_int ?Y"
  unfolding y_rep using dy by simp
have scale_z: "(of_int ?D :: rat) * z = of_int ?Z"
  unfolding z_rep using dz by simp

have scaled_equation:
  "((of_int ?D :: rat) * x)^2 =
    of_int A * ((of_int ?D :: rat) * y)^2 +
    of_int B * ((of_int ?D :: rat) * z)^2"
proof -
  have "((of_int ?D :: rat) * x)^2 = (of_int ?D :: rat)^2 * x^2"
    by (simp add: power2_eq_square)
  also have "... = (of_int ?D :: rat)^2 * (of_int A * y^2 + of_int B
* z^2)"
    using equation by simp
  also have "... = of_int A * ((of_int ?D :: rat) * y)^2 +
    of_int B * ((of_int ?D :: rat) * z)^2"
    by (simp add: power2_eq_square algebra_simps)
  finally show ?thesis .
qed
have rat_equation:
  "(of_int ?X :: rat)^2 = of_int A * (of_int ?Y :: rat)^2 +
    of_int B * (of_int ?Z :: rat)^2"
  using scaled_equation by (simp only: scale_x scale_y scale_z)
have cast_equation:
  "(of_int (?X^2) :: rat) = of_int (A * ?Y^2 + B * ?Z^2)"
  using rat_equation by simp
have int_equation: "?X^2 = A * ?Y^2 + B * ?Z^2"
  using cast_equation by (metis of_int_eq_iff)
have int_nonzero: "?X ≠ 0 ∨ ?Y ≠ 0 ∨ ?Z ≠ 0"
  using nonzero dx dy dz unfolding x_rep y_rep z_rep by auto
show ?thesis
  using int_nonzero int_equation by blast

```

```

qed

context ordered_sym_bibd
begin

lemma brc_sign_plus:
  assumes v_form:
    " $v = 4 * w + 1$ "
  shows
    " $(-1::int)^{(v - 1) \text{ div } 2} = 1$ "
proof -
  have exponent:
    " $(v - 1) \text{ div } 2 = 2 * w$ "
    using v_form
    by simp

  show ?thesis
    unfolding exponent
    by (simp add: power_mult)
qed

lemma brc_sign_minus:
  assumes v_form:
    " $v = 4 * w - 1$ "
  assumes w_pos:
    " $0 < w$ "
  shows
    " $(-1::int)^{(v - 1) \text{ div } 2} = -1$ "
proof -
  obtain s where w_eq:
    " $w = \text{Suc } s$ "
    using w_pos
    by (cases w) auto

  have v_eq:
    " $v = 4 * s + 3$ "
    using v_form w_eq
    by simp

  have exponent:
    " $(v - 1) \text{ div } 2 = 2 * s + 1$ "
    using v_eq
    by simp

  show ?thesis
    unfolding exponent
    by (simp add: power_add power_mult)
qed

```

```

lemma brc_plus_integer_solution_exists:
  assumes v_form:
    "v = 4 * w + 1"
  shows
    "∃ x y z :: int.
      (x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0) ∧
      x^2 =
        int (k - Λ) * y^2 +
        int Λ * z^2"
  proof -
    obtain x y z :: rat where nz:
      "x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0"
    and eq:
      "x^2 =
        of_nat (k - Λ) * y^2 +
        of_nat Λ * z^2"
    using brc_plus_rational_solution_exists[
      OF v_form]
    by blast

    have eq_cast:
      "x^2 =
        of_int (int (k - Λ)) * y^2 +
        of_int (int Λ) * z^2"
      using eq
      by simp

    show ?thesis
      using rat_quadratic_solution_to_int[
        where A = "int (k - Λ)"
        and B = "int Λ"
        and x = x and y = y and z = z,
        OF nz eq_cast]
      .
  qed

```

```

lemma brc_minus_integer_solution_exists:
  assumes v_form:
    "v = 4 * w - 1"
  assumes w_pos:
    "0 < w"
  shows
    "∃ x y z :: int.
      (x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0) ∧
      x^2 =
        int (k - Λ) * y^2 -
        int Λ * z^2"
  proof -
    obtain x y z :: rat where nz:

```

```

"x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0"
and eq:
"x^2 =
  of_nat (k - Λ) * y^2 -
  of_nat Λ * z^2"
using brc_minus_rational_solution_exists[
  OF v_form w_pos]
by blast

have eq_cast:
"x^2 =
  of_int (int (k - Λ)) * y^2 +
  of_int (- int Λ) * z^2"
using eq
by simp

obtain X Y Z :: int where nz_int:
"x ≠ 0 ∨ Y ≠ 0 ∨ Z ≠ 0"
and eq_int:
"X^2 =
  int (k - Λ) * Y^2 +
  (- int Λ) * Z^2"
using rat_quadratic_solution_to_int[
  where A = "int (k - Λ)"
  and B = "- int Λ"
  and x = x and y = y and z = z,
  OF nz eq_cast]
by blast

have eq_minus:
"X^2 =
  int (k - Λ) * Y^2 -
  int Λ * Z^2"
using eq_int
by simp

show ?thesis
using nz_int eq_minus
by blast
qed

```

Every odd natural number is congruent to either 1 or 3 modulo 4. The preceding results supply an integer solution with a plus sign in the first case and a minus sign in the second. The corresponding power of -1 rewrites these two equations as one uniform statement.

```

theorem bruck_ryser_chowla_odd:
  assumes odd_v:
    "odd v"
  shows

```

```

    "∃ x y z :: int.
      (x ≠ 0 ∨ y ≠ 0 ∨ z ≠ 0) ∧
      x^2 =
        int (k - Λ) * y^2 +
        (-1::int)^(v - 1) div 2) *
        int Λ * z^2"
  proof -
    have "v mod 4 = 1 ∨ v mod 4 = 3"
      using odd_v by presburger
    then show ?thesis
      proof
        assume "v mod 4 = 1"
        then obtain w where v_form: "v = 4 * w + 1"
          using brc_v_eq_4w_plus_1 by blast
        show ?thesis
          using brc_plus_integer_solution_exists[OF v_form]
            unfolding brc_sign_plus[OF v_form]
              by simp
      next
        assume "v mod 4 = 3"
        then obtain w where w_pos: "0 < w" and v_form: "v = 4 * w - 1"
          using brc_v_eq_4w_minus_1 by blast
        show ?thesis
          using brc_minus_integer_solution_exists[OF v_form w_pos]
            unfolding brc_sign_minus[OF v_form w_pos]
              by simp
      qed
    qed
  end
end

```

References

- [1] Douglas R. Stinson, *Combinatorial Designs: Constructions and Analysis*, Springer, New York, 2004.